

Lecture 3: Approximation Theory

(or: what functions can we approximate with a neural network?)

Speaker: Phillip Isola

Includes slides adapted from: Stefanie Jegelka, Jeremy Bernstein, Francois Fleuret, Omar Khattab

3. What functions can neural networks express?

- Approximation theory (and why we care)
- What kinds of functions are ReLU networks?
- Universal approximation with 3-layer ReLU networks
- Width versus depth

New demos under course materials!

MIT 6.7960 — APPROXIMATION THEORY

Overview

From one ReLU to learned functions, network shape, and richer inputs.

01 — Approximation

Fit a 1D target with a two-layer ReLU MLP. Slide H to see how more hidden units improve the piecewise-linear approximation.

02 — Effect of ReLU on Kinks

Shift different functions with bias and see how ReLU creates or removes kinks, including on the Telgarsky sawtooth.

03 — Width vs Depth

Fix the total neuron budget N , then vary the aspect ratio (width/depth). Compare wide-shallow vs narrow-deep networks on the same task.

04 — Sinusoidal Positional Encoding

Side-by-side: plain MLP vs pos-enc MLP fitting a noisy scatter plot. Drag H and L to see how encoding unlocks high-frequency fitting.

Approximation, Optimization, and Generalization

- We minimize the *empirical risk*: $\hat{\mathcal{R}}(\theta) = \frac{1}{N} \mathcal{L}(f_{\theta}(\mathbf{x}^{(i)}), \mathbf{y}^{(i)})$
- We actually want the *population risk (test error)* to be small:

$$\mathcal{R}(\theta) = \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim \mathcal{P}} \mathcal{L}(f_{\theta}(\mathbf{x}), \mathbf{y})$$

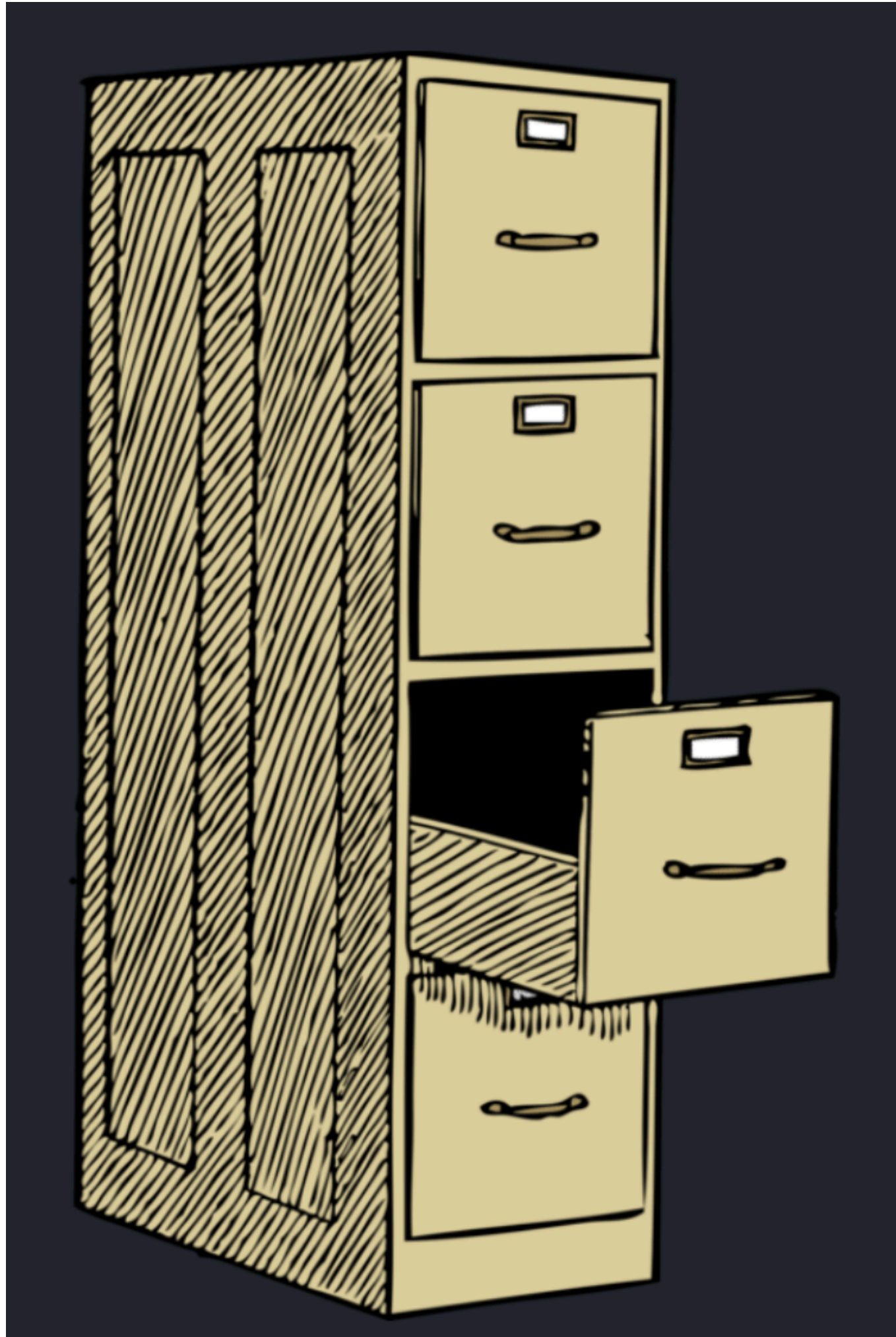
Important questions:

Approximation: what is the best $\hat{\mathcal{R}}(\theta^*)$ we can achieve with our model?

Optimization: how well are we minimizing $\hat{\mathcal{R}}(\theta)$?

Generalization: how different is $\hat{\mathcal{R}}(\theta)$ from $\mathcal{R}(\theta)$?

The filing cabinet model



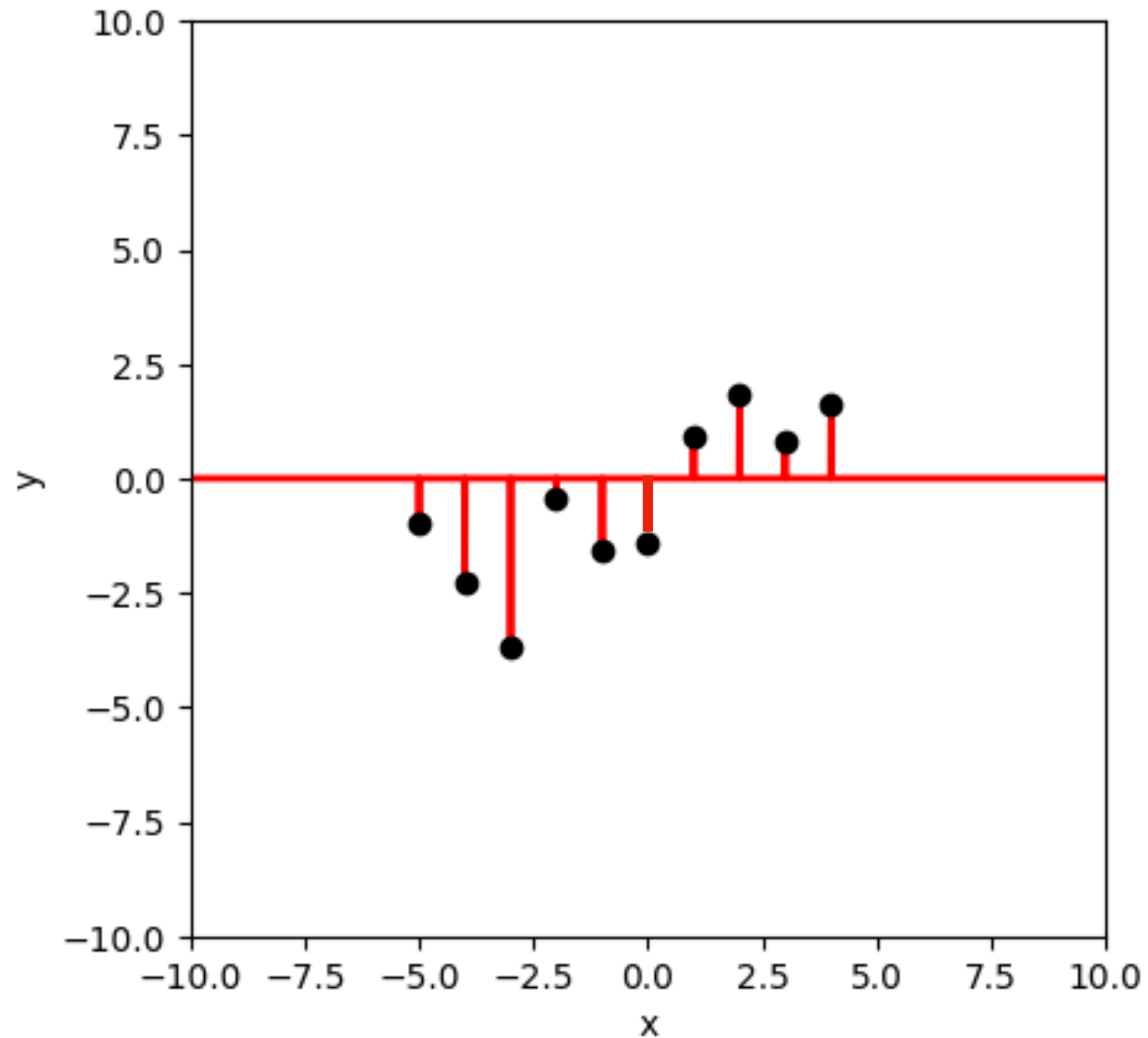
Every time we see a new training point (x,y) , we put it in the cabinet.

```
def train(X,Y):  
    for x, y in zip(X, Y):  
        cabinet[x] = y
```

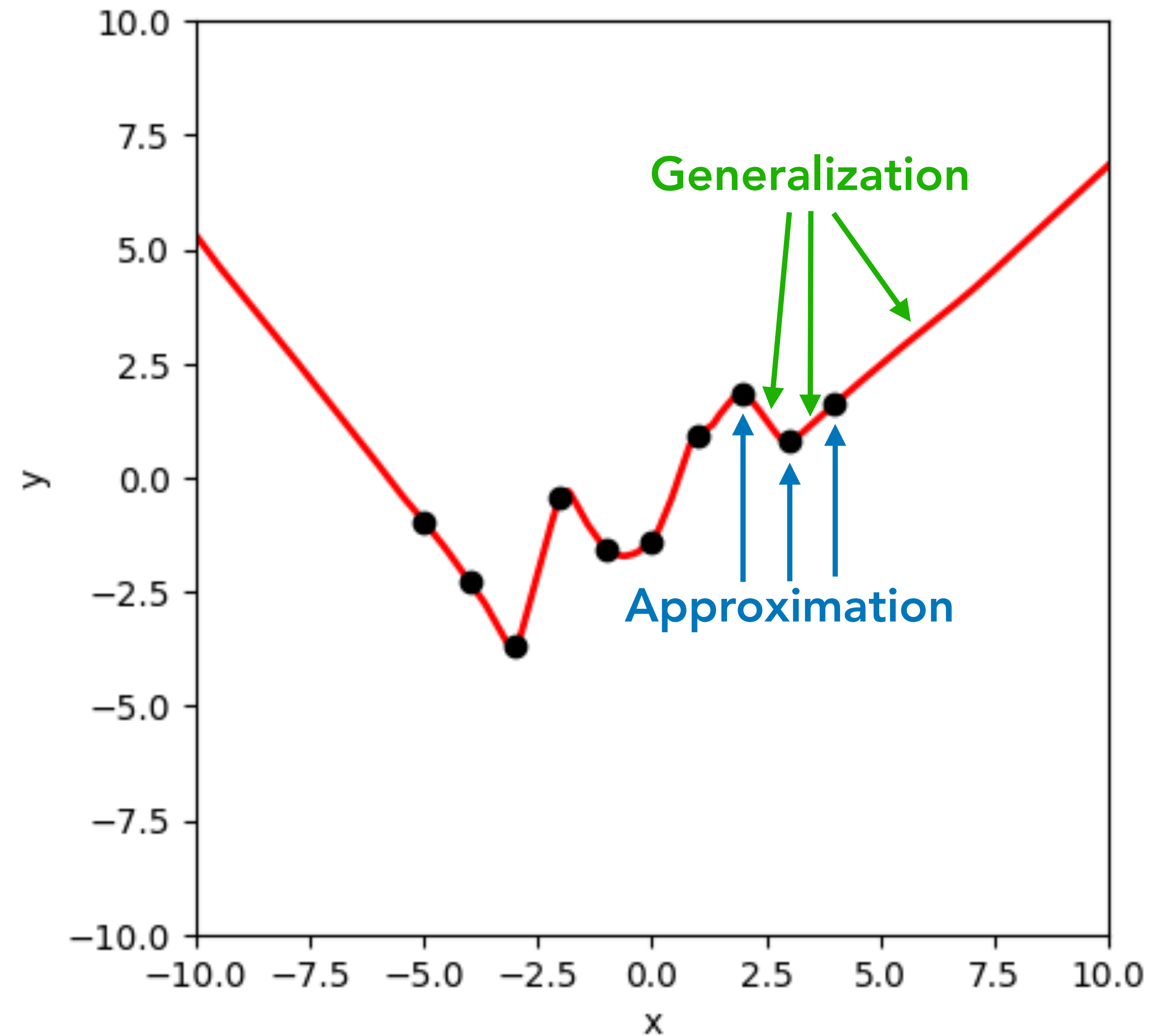
```
def predict(x):  
    if x in cabinet:  
        return cabinet[x]  
    else:  
        return 0
```

Approximation vs Generalization

Filing cabinet



3-layer relu-MLP



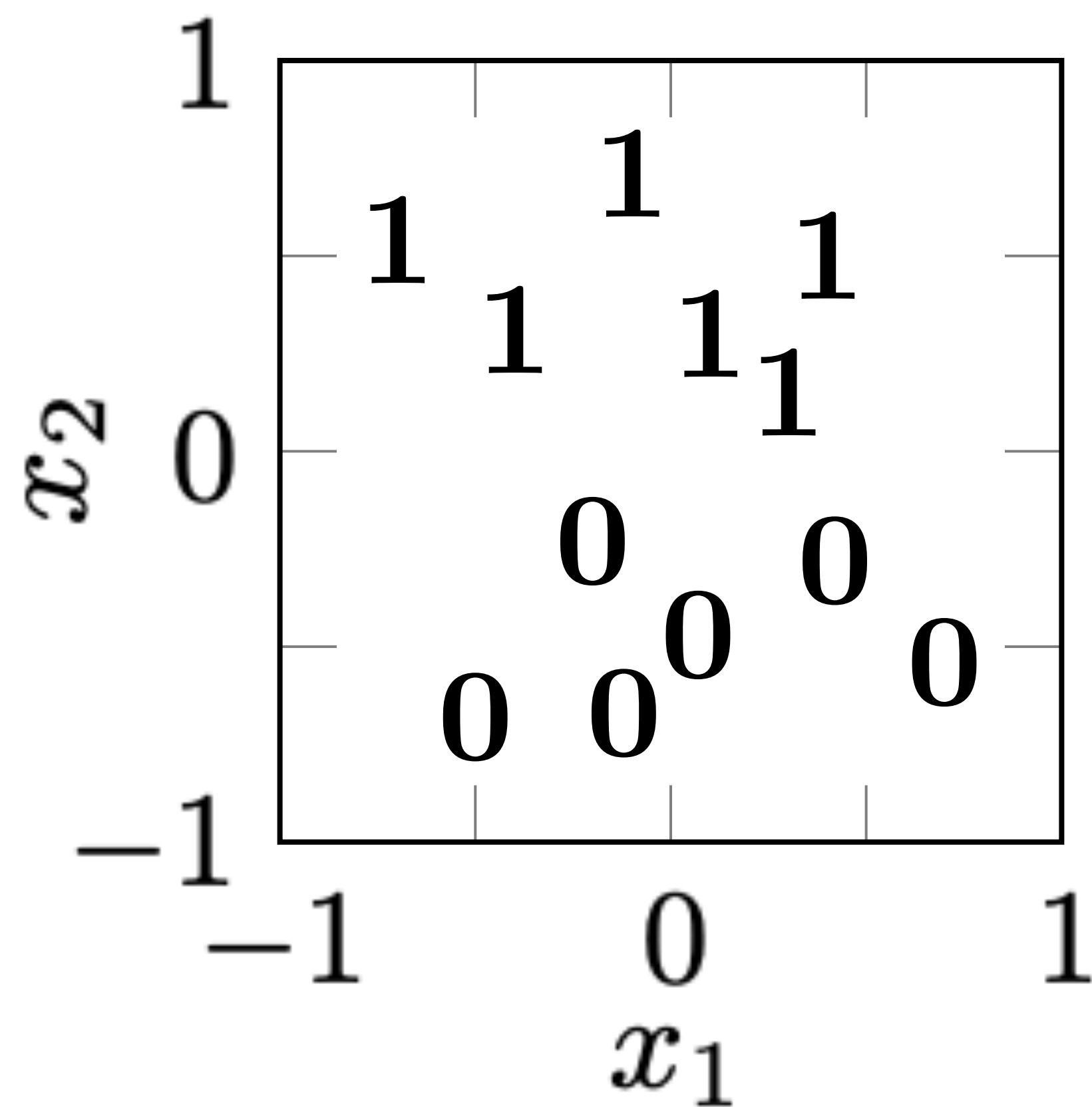
Approximation can be trivial.

What is interesting is:

- Approximation with a function class that generalizes.
- Approximation with few parameters.

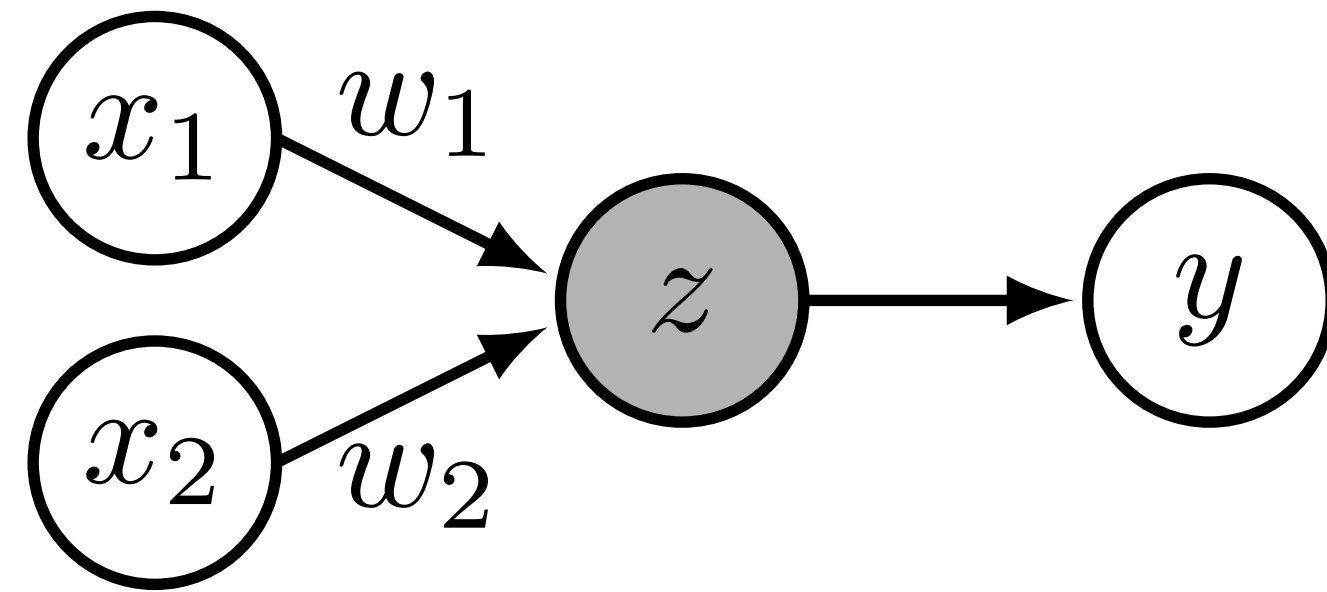
We are interested in statements like: “Nice arch X can approx important function class $Y(N)$ with $O(\log N)$ neurons” (where N is some parameter of the function class, such as its smoothness)

Training data



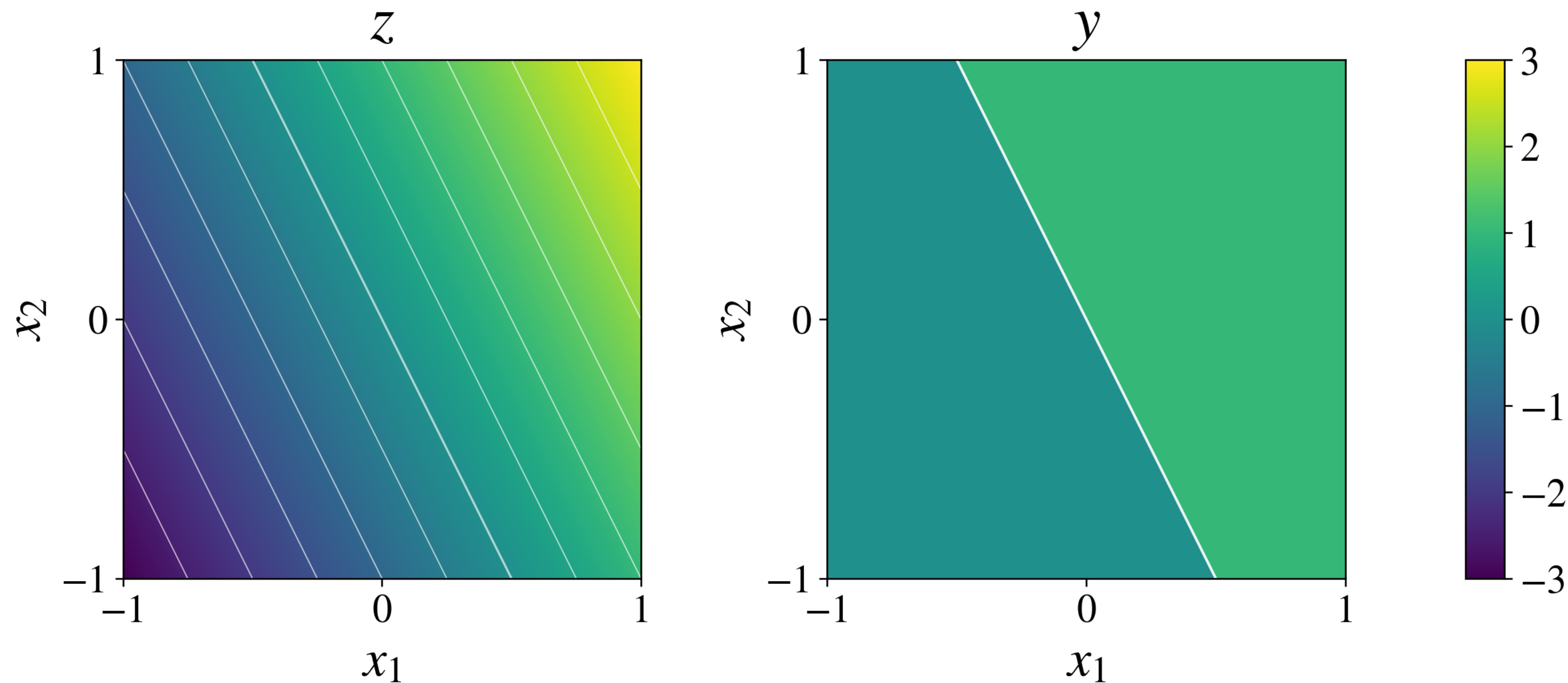
$$f(\mathbf{x}) = \text{threshold}(\mathbf{w}^\top \mathbf{x} + \mathbf{b})$$

Example: linear classification with a 1-layer net



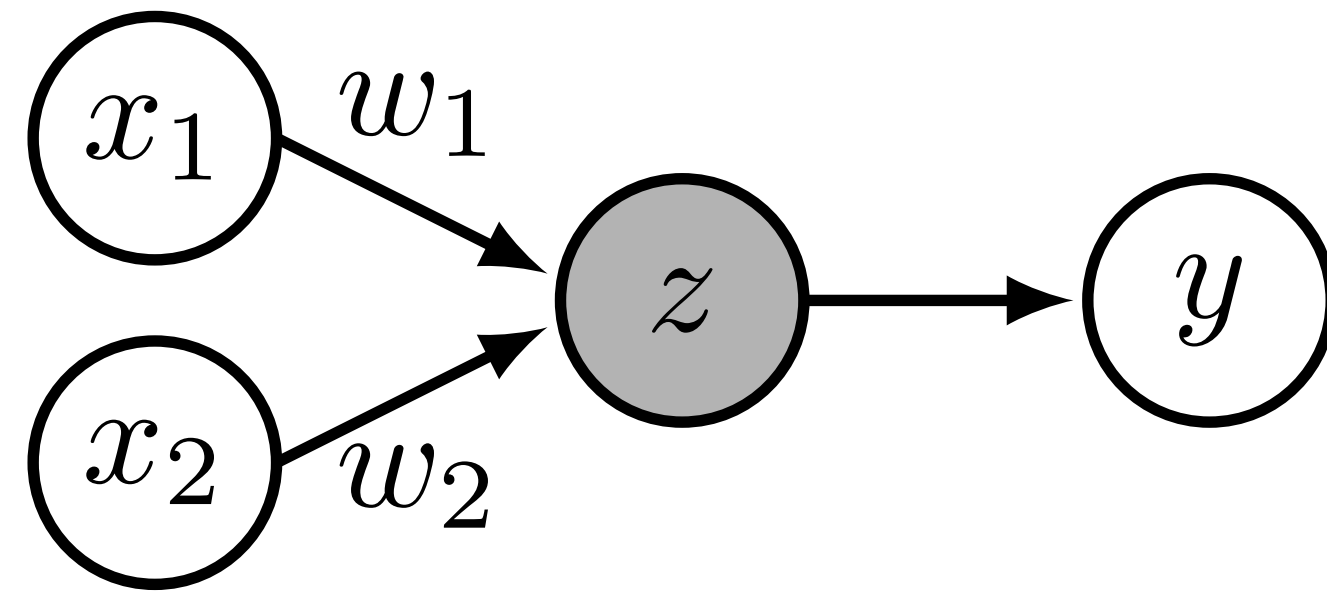
$$z = \mathbf{x}^T \mathbf{w} + b$$

$$y = g(z)$$



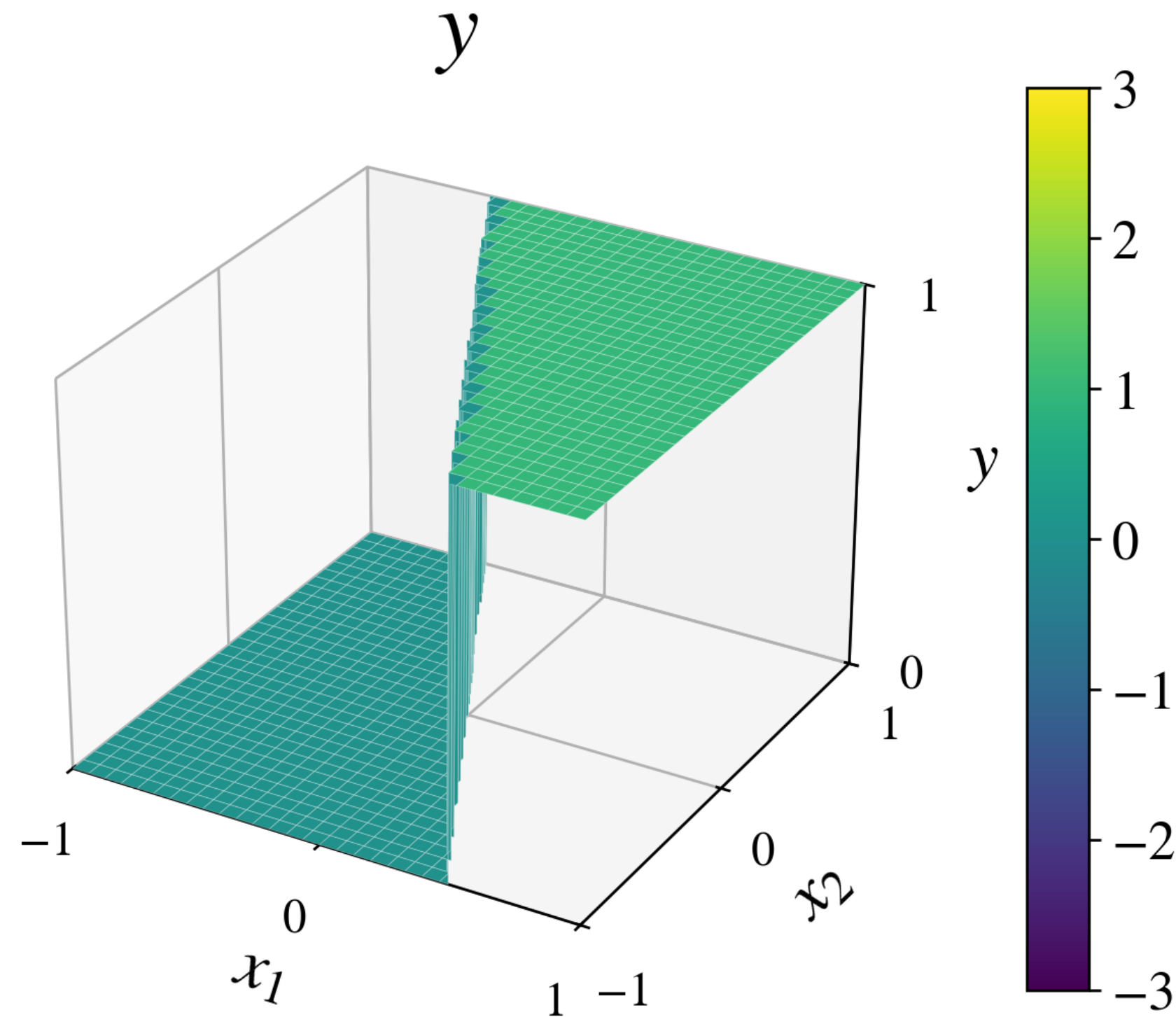
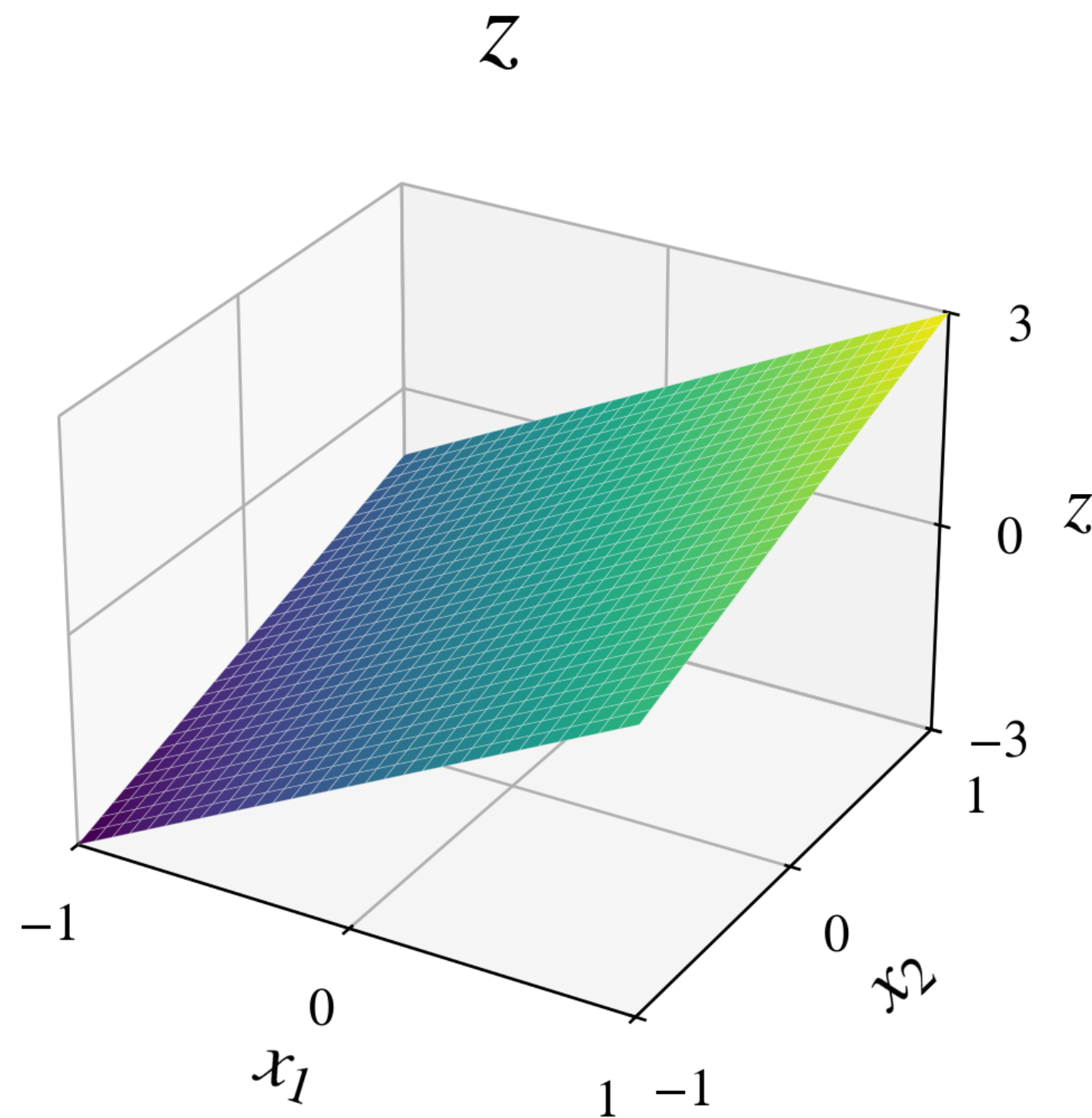
One layer neural net (perceptron) can perform linear classification!

Example: linear classification with a 1-layer net



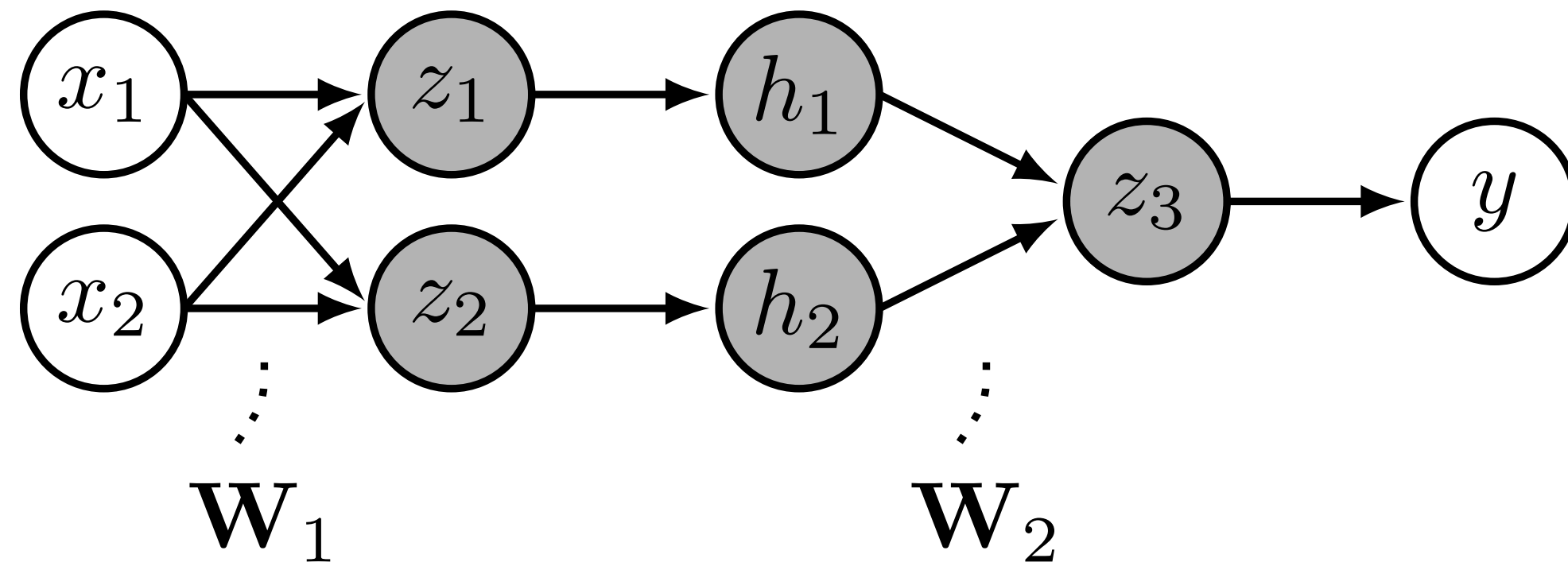
$$z = \mathbf{x}^T \mathbf{w} + b$$

$$y = g(z)$$



One layer neural net (perceptron) can perform linear classification!

Example: nonlinear classification with a deep net

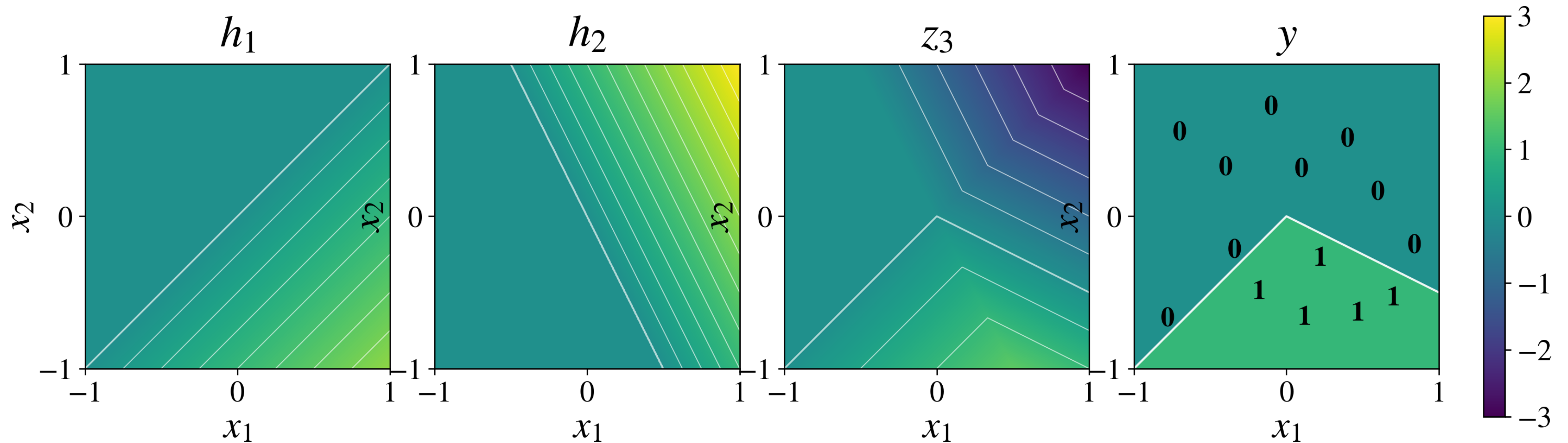


$$\mathbf{z} = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1$$

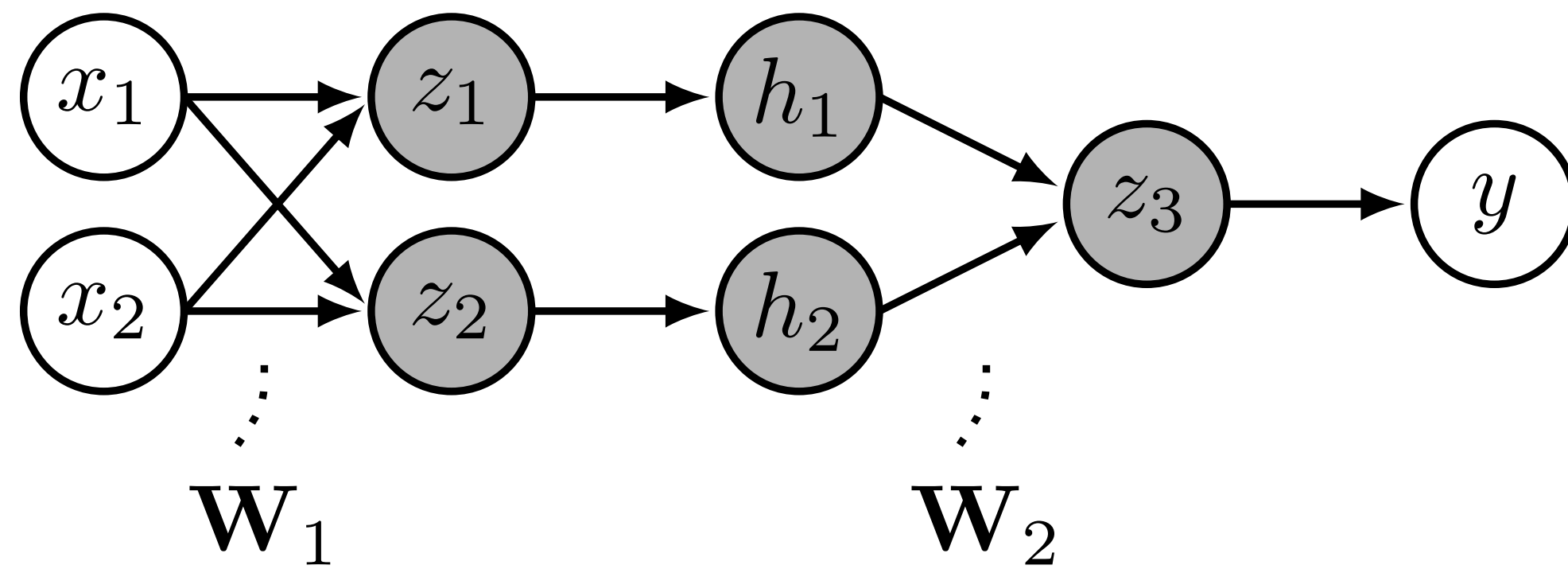
$$\mathbf{h} = g(\mathbf{z})$$

$$z_3 = \mathbf{W}_2 \mathbf{h} + b_2$$

$$y = 1(z_3 > 0)$$



Example: nonlinear classification with a deep net

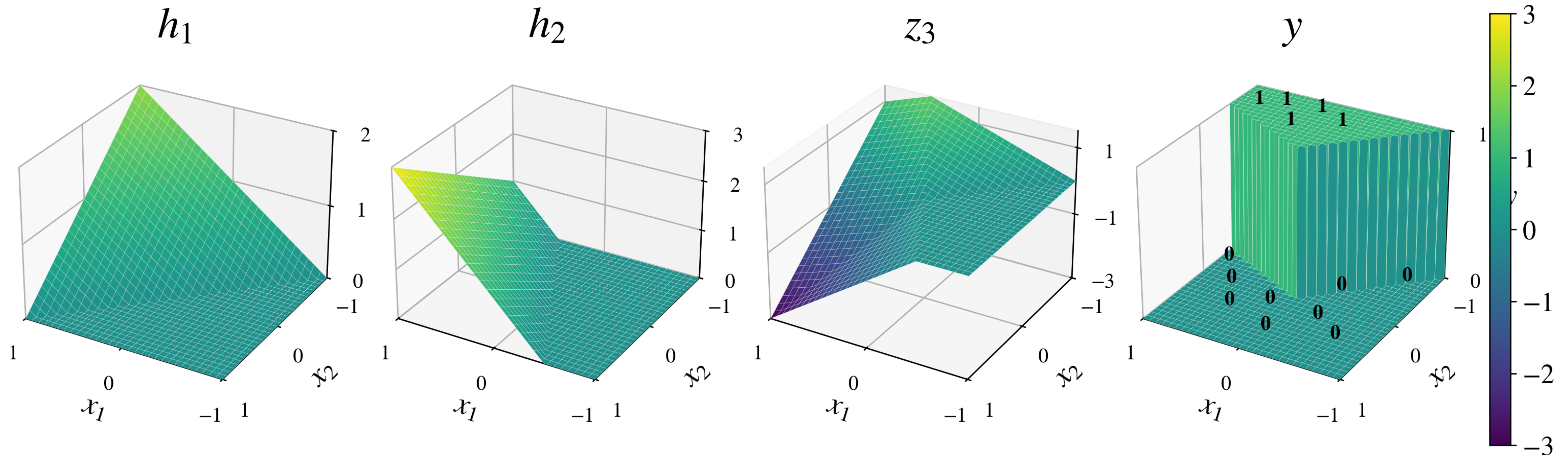


$$\mathbf{z} = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1$$

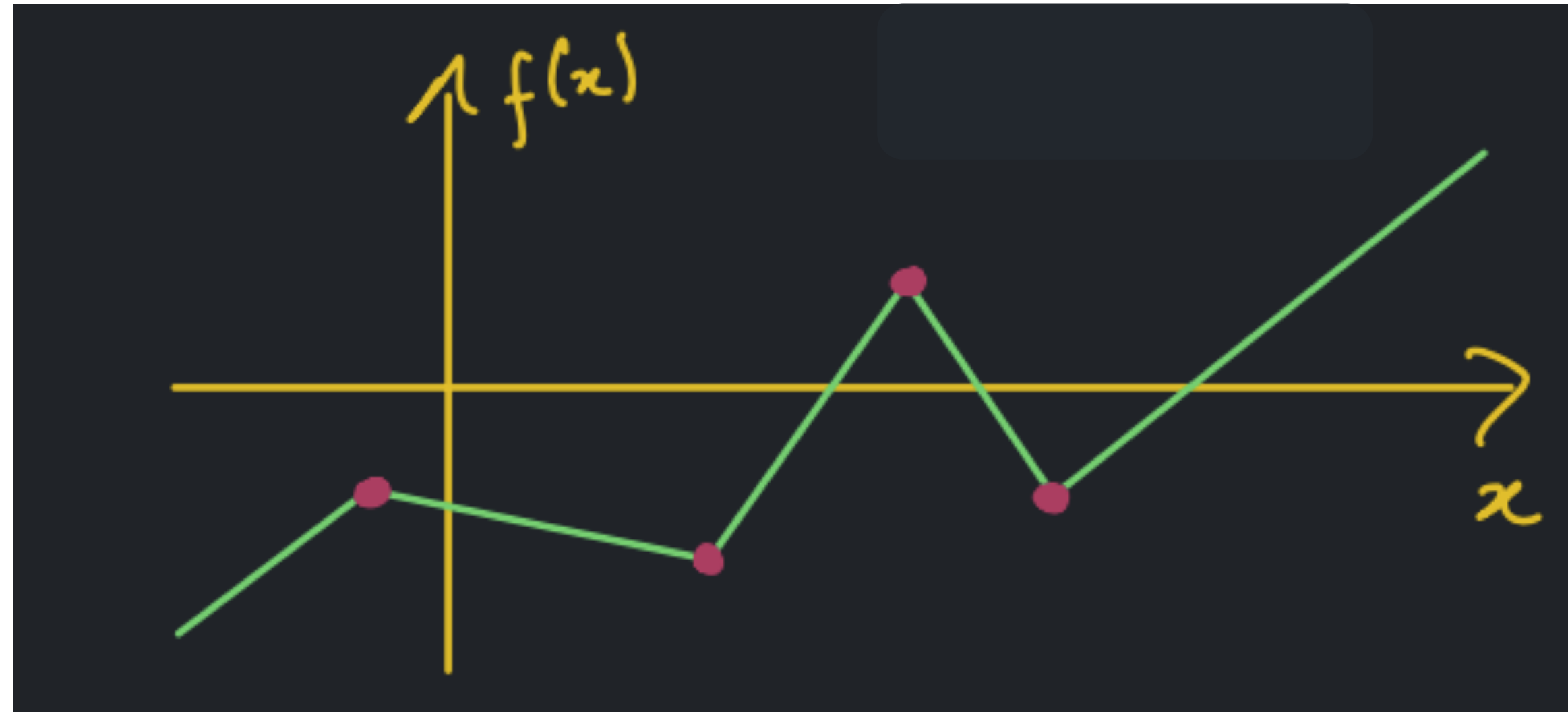
$$\mathbf{h} = g(\mathbf{z})$$

$$z_3 = \mathbf{W}_2 \mathbf{h} + b_2$$

$$y = 1(z_3 > 0)$$



Piecewise linear ("PWL") functions



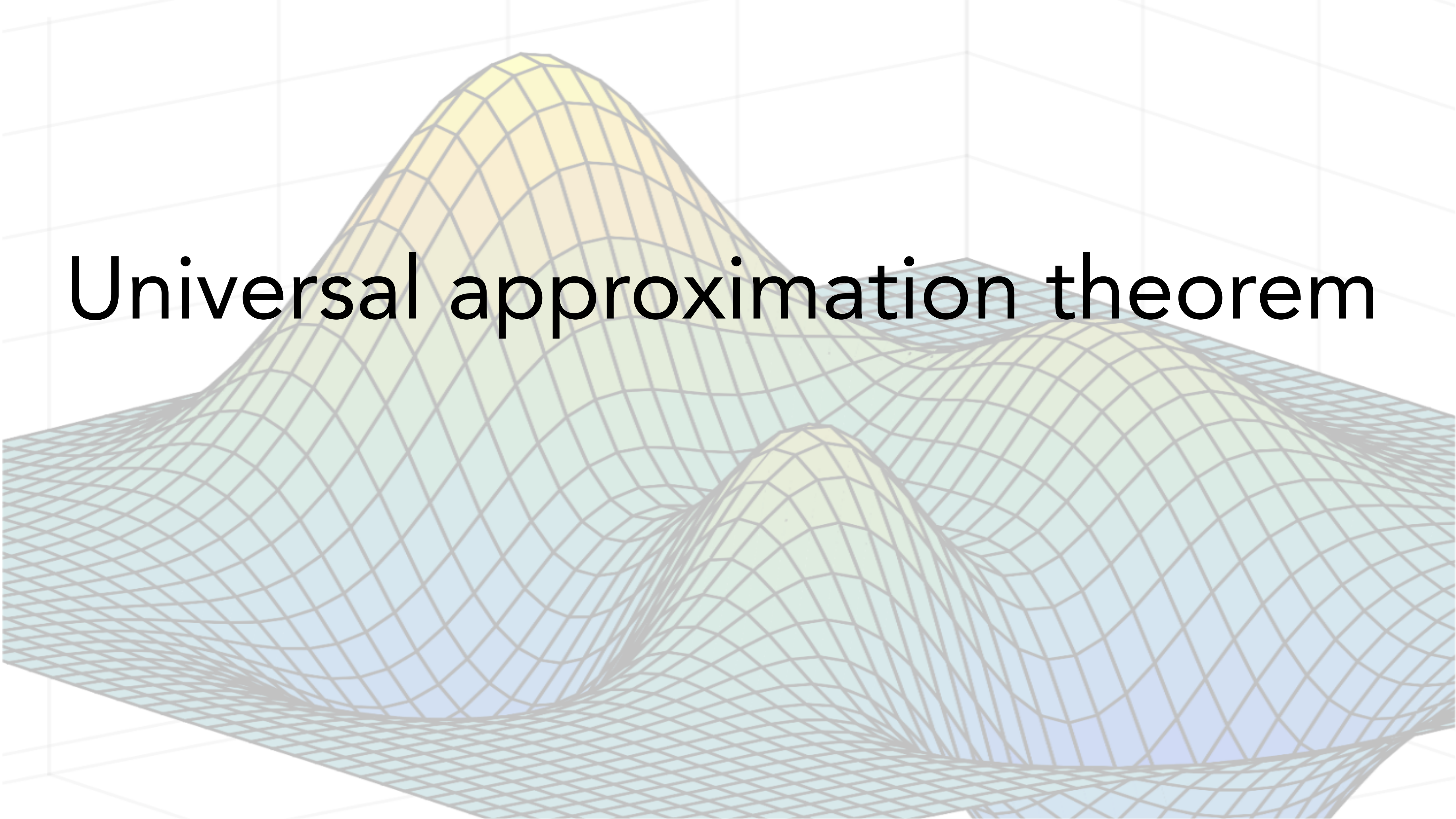
Claim: ReLU networks are PWL. Why?

1. Linear layers are are PWL. /
2. ReLU is PWL. ---/
3. f, g both PWL $\Rightarrow f \circ g$ is PWL

Main question: **what class of functions can we express with deep networks?**

Answer: **Everything (that is continuous)**

Universal approximation theorem



Formalizing the Approximation problem

Given a family of curves G and a family of neural networks F .

For any curve $g \in G$, is there a neural net $f \in F$ such that $\text{error}(f, g) < \varepsilon$?

Theorem (Universal approximation with 2 layers, *Hornik, Stinchcombe, White 1989; Leshno et al. 1993*):

Suppose we use a continuous activation function that is not a polynomial. Then for any continuous function $g : \mathbb{R}^d \rightarrow \mathbb{R}$ and any error $\varepsilon > 0$, there exists a 2-layer network f , with bias terms, such that

$$|f(\mathbf{x}) - g(\mathbf{x})| \leq \varepsilon \quad \text{for all } \mathbf{x} \in [0, 1]^d$$

Two-layer MLPs are “universal approximators”! We’ll informally prove a weaker result today.

Consider not all continuous functions, but specifically L-Lipschitz functions.

Basically, functions g that cannot change faster than slope L . The output difference is bounded by a linear factor of input distance.

Theorem. Let g be an arbitrary L -Lipschitz function and $\varepsilon > 0$. Then there exists a 3-layer ReLU network f with $O((L/\varepsilon)^d)$ units such that

$$\int_{[0,1]^d} |f(\mathbf{x}) - g(\mathbf{x})| d\mathbf{x} \leq 2\varepsilon.$$

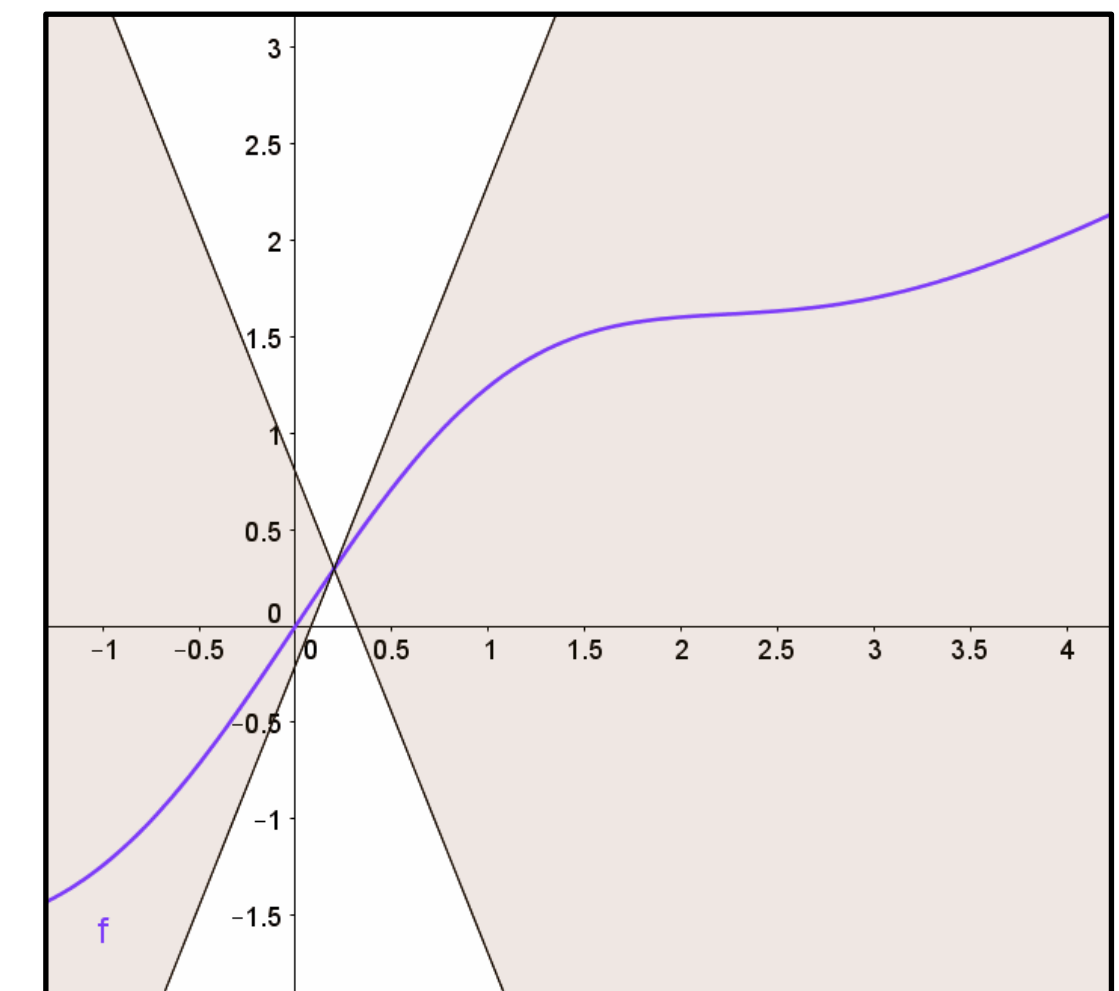
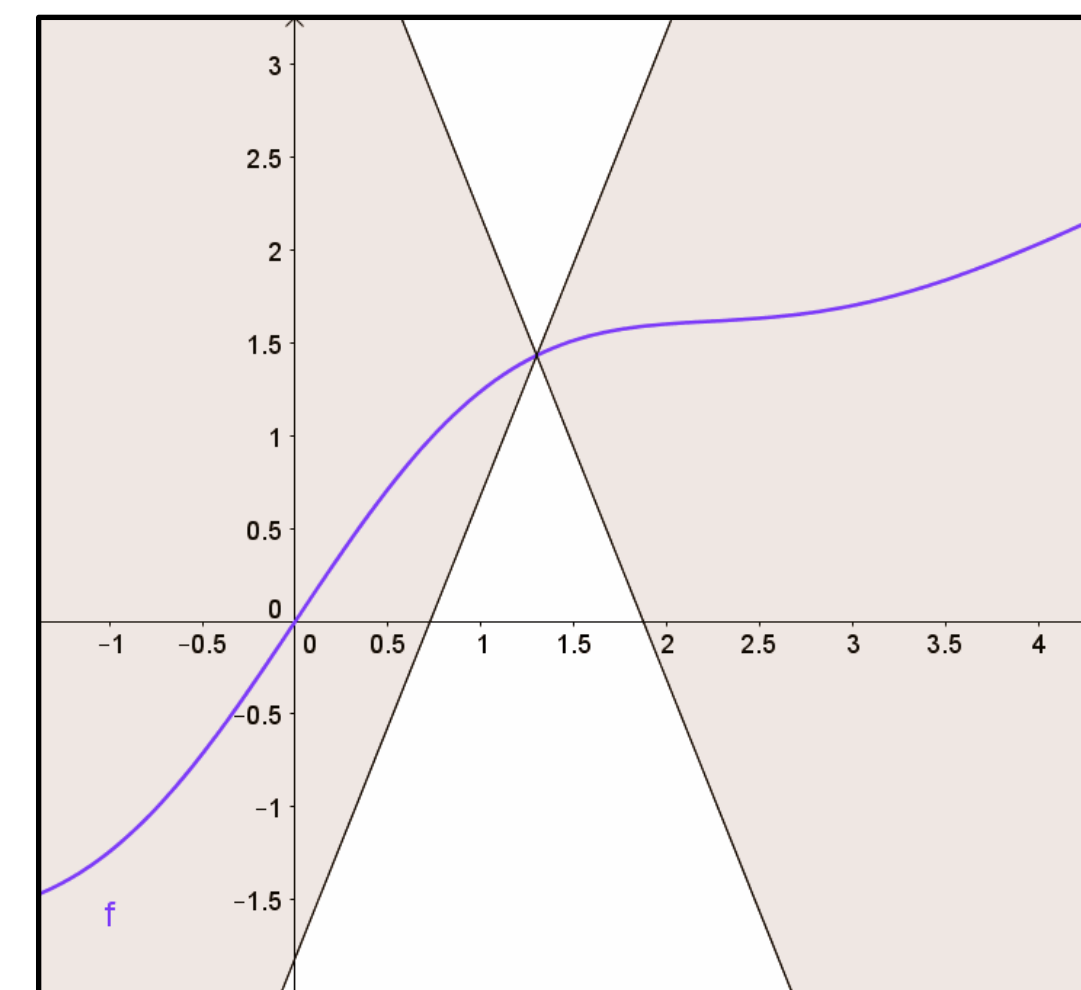
Definition (Lipschitz continuity). A function $g : \mathbb{R}^d \rightarrow \mathbb{R}$ is called L -Lipschitz, with constant $L \geq 0$, if

$$|g(\mathbf{x}) - g(\mathbf{x}')| \leq L \|\mathbf{x} - \mathbf{x}'\|$$

Here, $\|\mathbf{x} - \mathbf{x}'\|$ denotes the *norm* (distance), e.g. ℓ_∞ or Euclidean. We’ll use the former today.

$$\|\mathbf{x} - \mathbf{x}'\|_\infty := \max_{1 \leq i \leq d} |x_i - x'_i|.$$

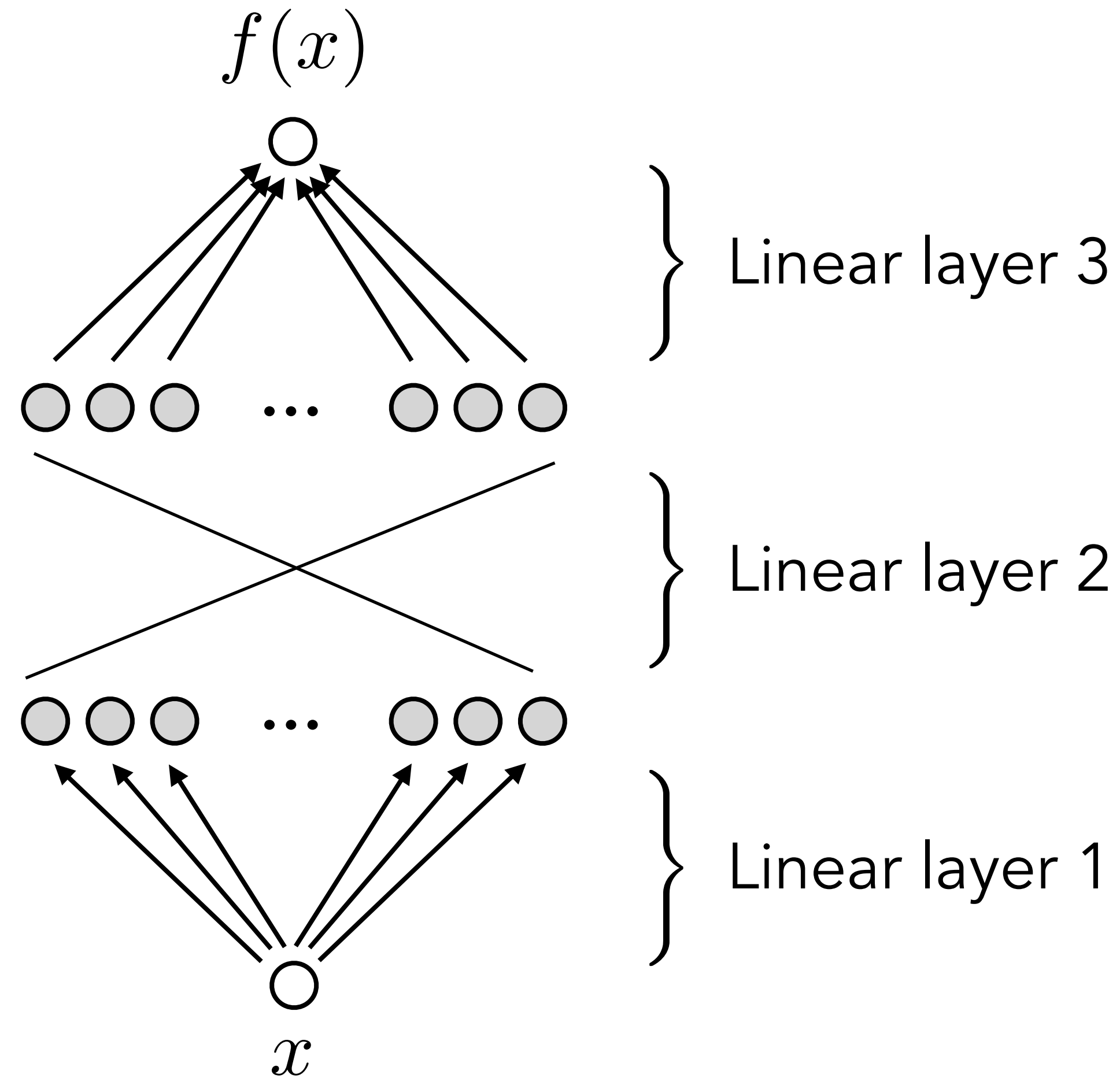
$$\|\mathbf{x} - \mathbf{x}'\|_2 = \sqrt{\sum_{i=1}^d (x_i - x'_i)^2}$$



Approximating a univariate function with an MLP

$$f : \mathbb{R} \rightarrow \mathbb{R}$$

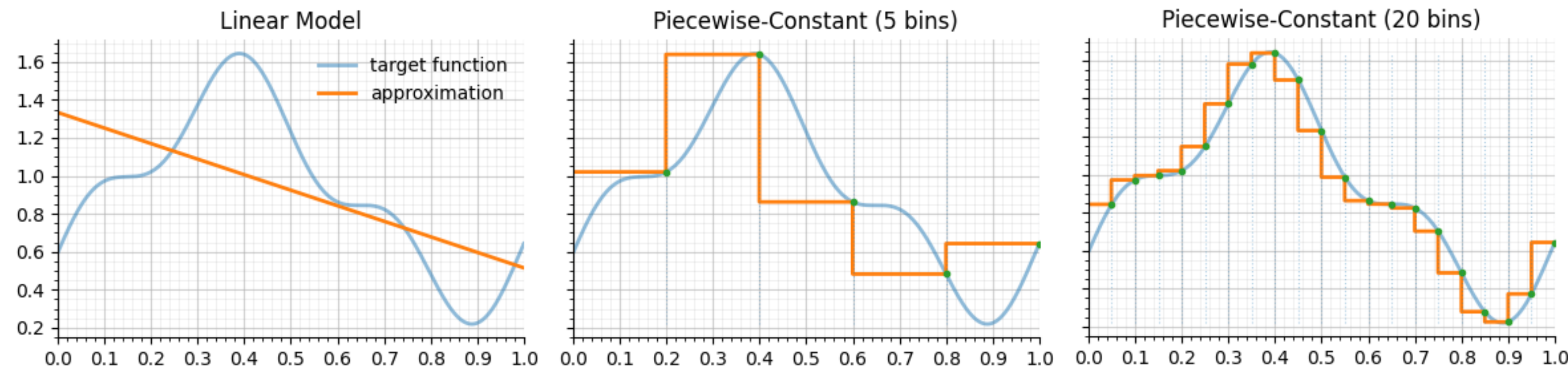
"3-Layer" ReLU network



Proof idea

1. Approximate univariate continuous functions by indicator functions
2. Use ReLUs to build continuous approximation to indicator functions (takes two layers)
3. Combine linearly such approx indicator functions using layer 3

To start simple, let's approximate a univariate $g: \mathbb{R} \rightarrow \mathbb{R}$ first.



```
def approx(x): # 5 bins
    if x < 0.2: return 1.02 # ≈ f(0.2)
    elif x < 0.4: return 1.64 # ≈ f(0.4)
    elif x < 0.6: return 0.86 # ≈ f(0.6)
    elif x < 0.8: return 0.48 # ≈ f(0.8)
    else: return 0.64 # ≈ f(1.0)
```

How many bins (or **if** statements) should we create so the error is small?

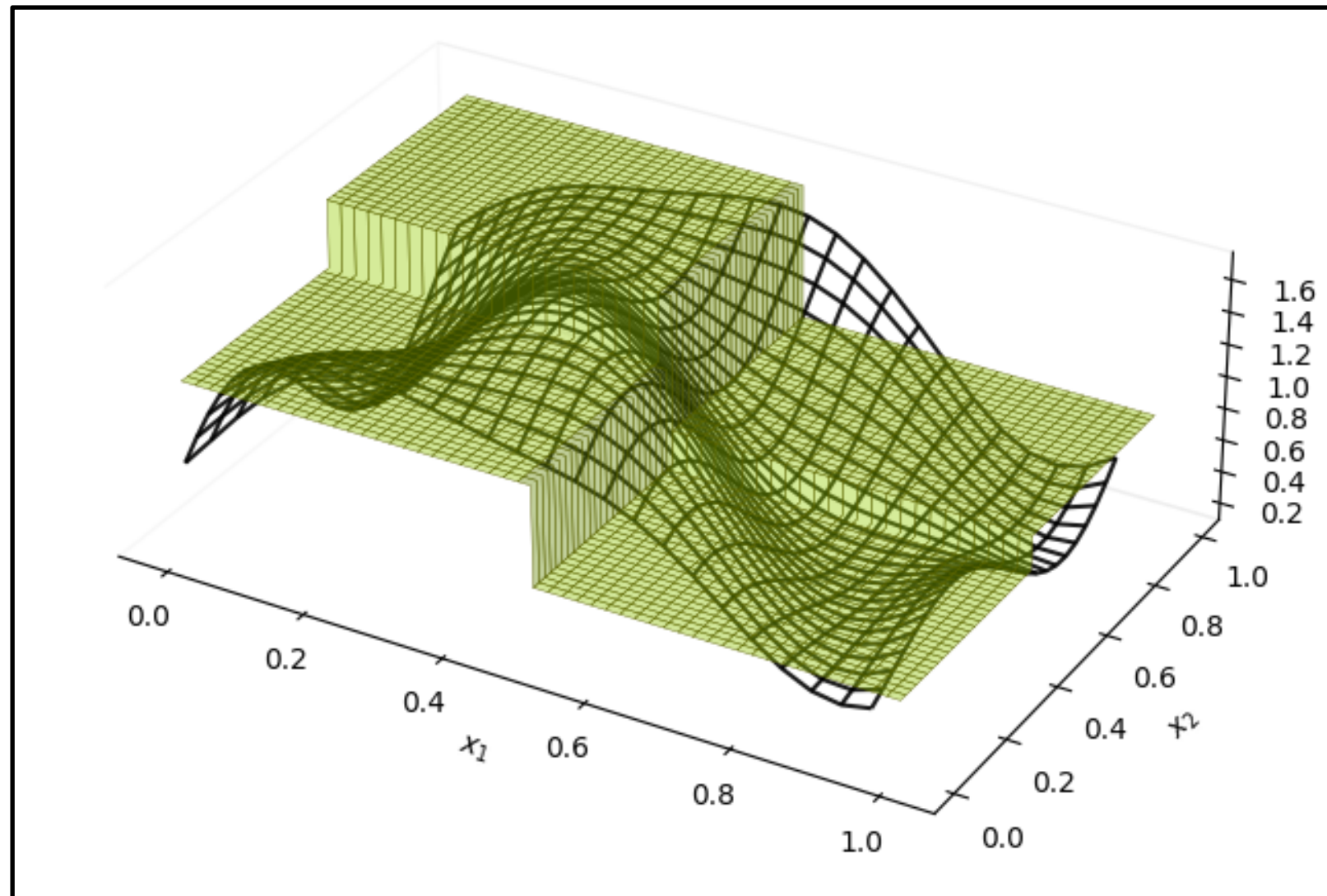
Because g is L -Lipschitz, moving the input by Δx can change g by at most $L|\Delta x|$.

So if each bin has width $\Delta x \leq \varepsilon / L$, then g can change by at most ε within a bin.

Thus, $N = L / \varepsilon$ bins can guarantee ε error (at any input! and thereby in the integral from 0 to 1 too)

Wait. If a basic Python program with **if** statements does it, why do we need MLPs...?

This works in higher dimensions, but requires exponentially-many bins.



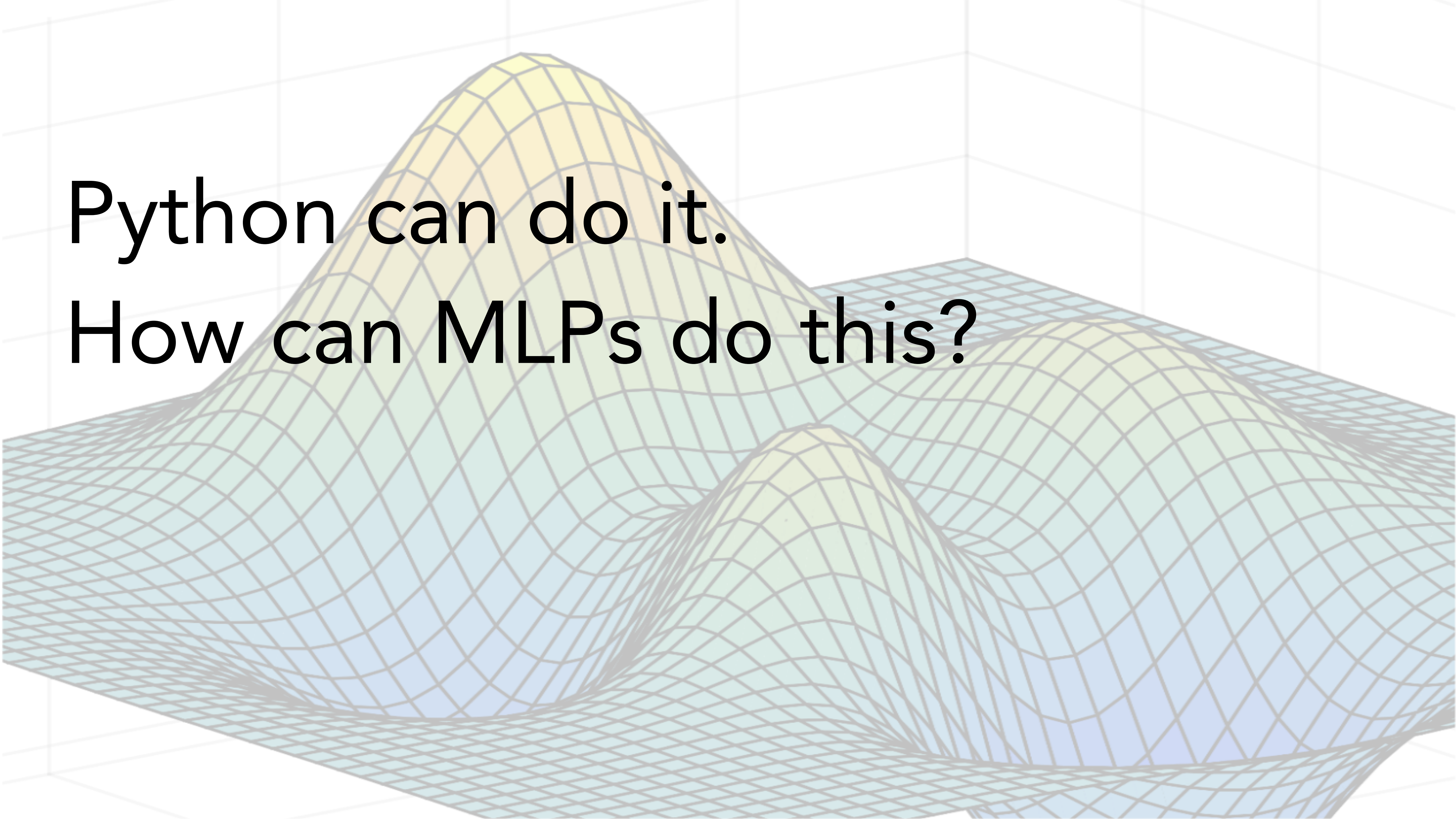
```
def approx(x1, x2): # 2x2 bins
    if (0 < x1 < 0.5) and (0 < x2 < 0.5): return 1.18
    if (0 < x1 < 0.5) and (0.5 < x2 < 1.0): return 1.60
    if (0.5 < x1 < 1.0) and (0 < x2 < 0.5): return 0.56
    if (0.5 < x1 < 1.0) and (0.5 < x2 < 1.0): return 0.77
```

How many bins (or **if** statements) should we create?

Because g is L -Lipschitz, we still want each bin to have side length $\Delta x \leq \varepsilon / L$ in each dimension.

Using the ℓ_∞ norm, error $|g(\mathbf{x}) - g(\mathbf{x}')| \leq L\|\mathbf{x} - \mathbf{x}'\| \leq L \cdot \varepsilon / L = \varepsilon$.

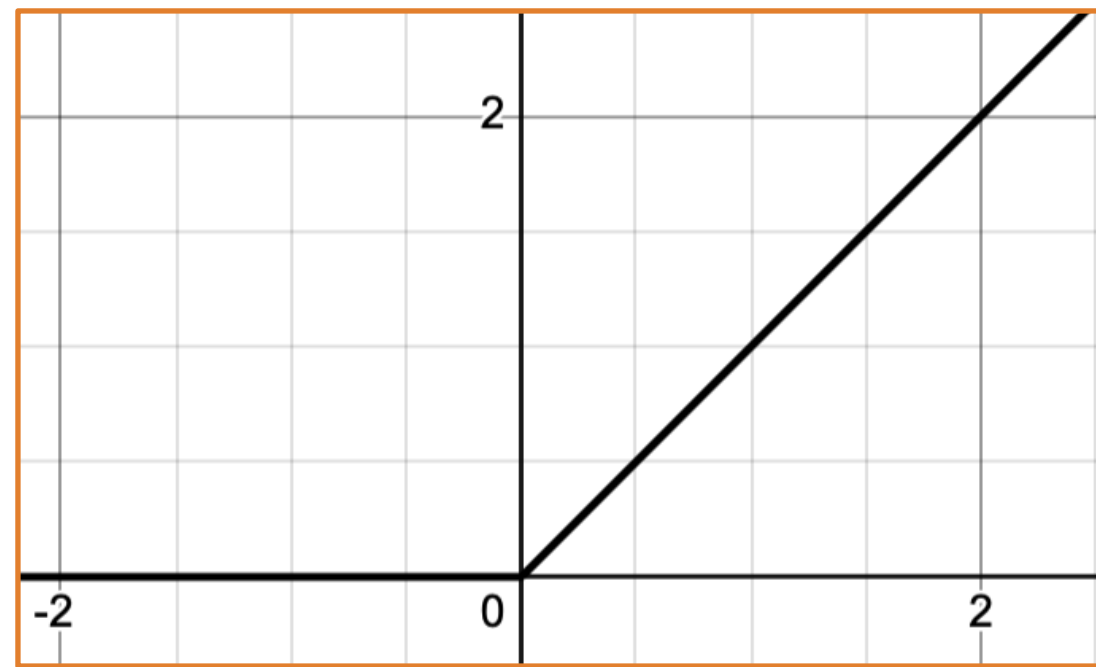
For the step size to be ε / L , $N = (L / \varepsilon)^d$ bins are needed overall.



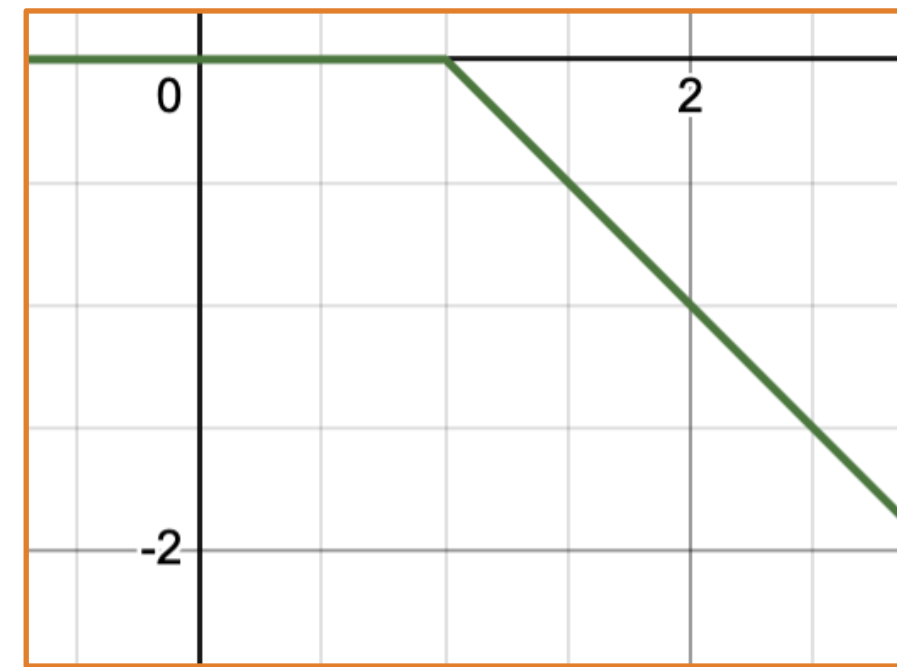
Python can do it.
How can MLPs do this?

Let's prove that a really wide MLP can (approximately!) do the same thing!

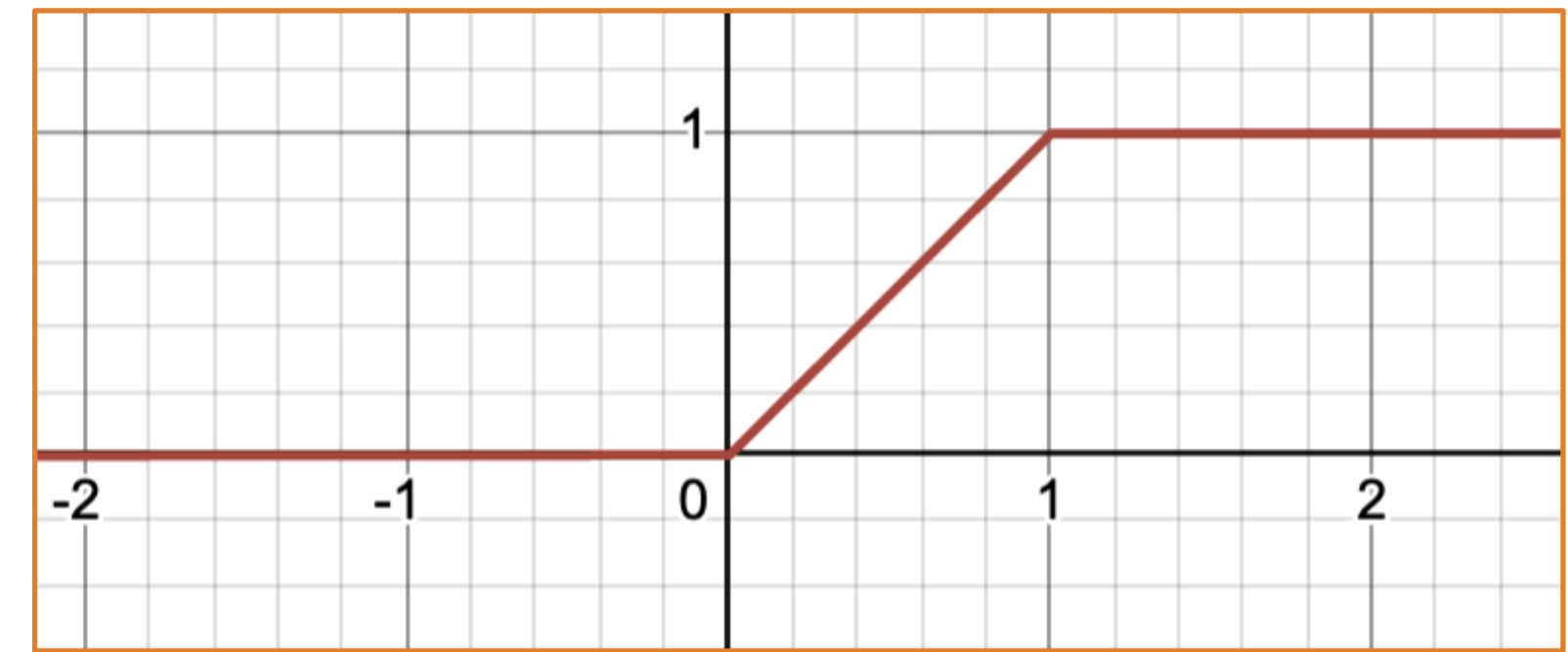
For a single-dimensional input, can we express a conditional like **if** $u \leq x < v$: **return** w with some ReLU compositions? Basically, we want a block of height w from $x=u$ to $x=v$.



$\text{ReLU}(x)$



$-\text{ReLU}(x-1)$



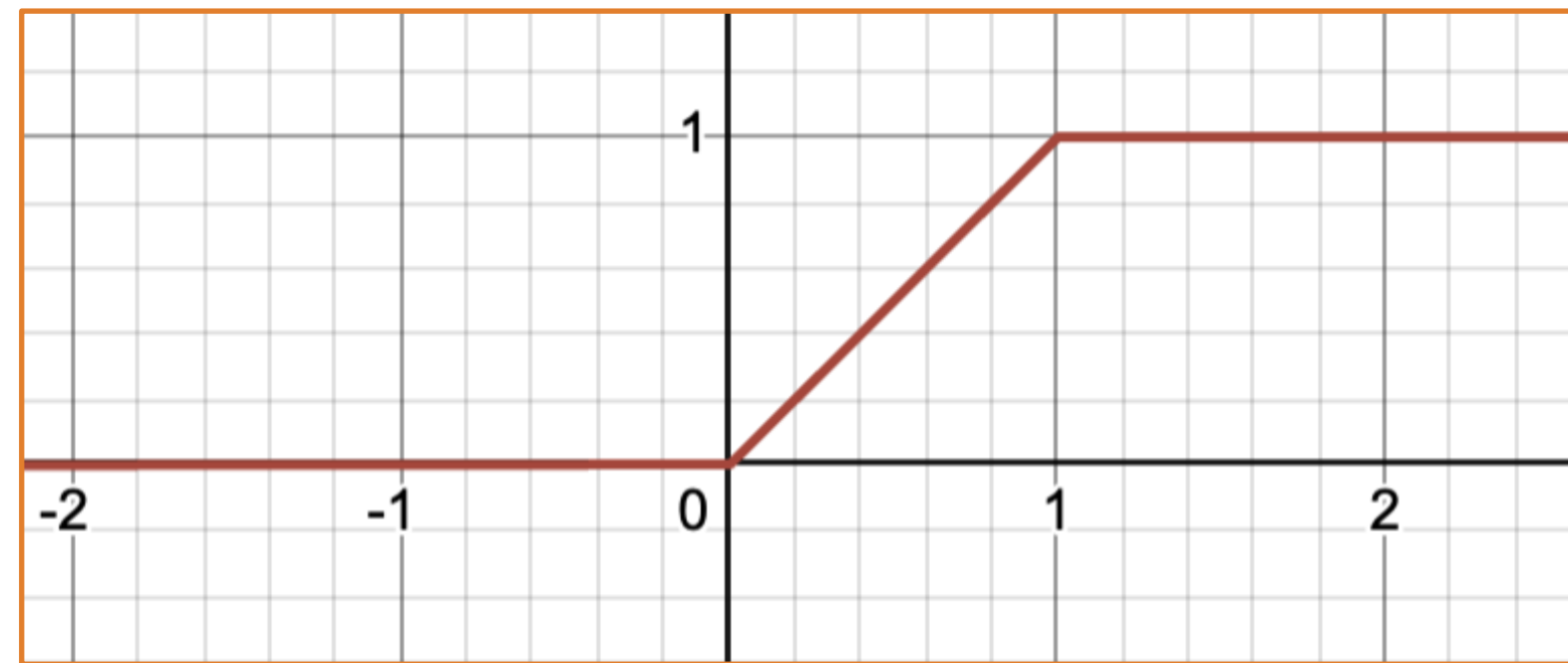
$\text{ReLU}(x) - \text{ReLU}(x-1)$

Define $U(x, t, g) = \text{ReLU}(x - t) - \text{ReLU}(x - t - g)$

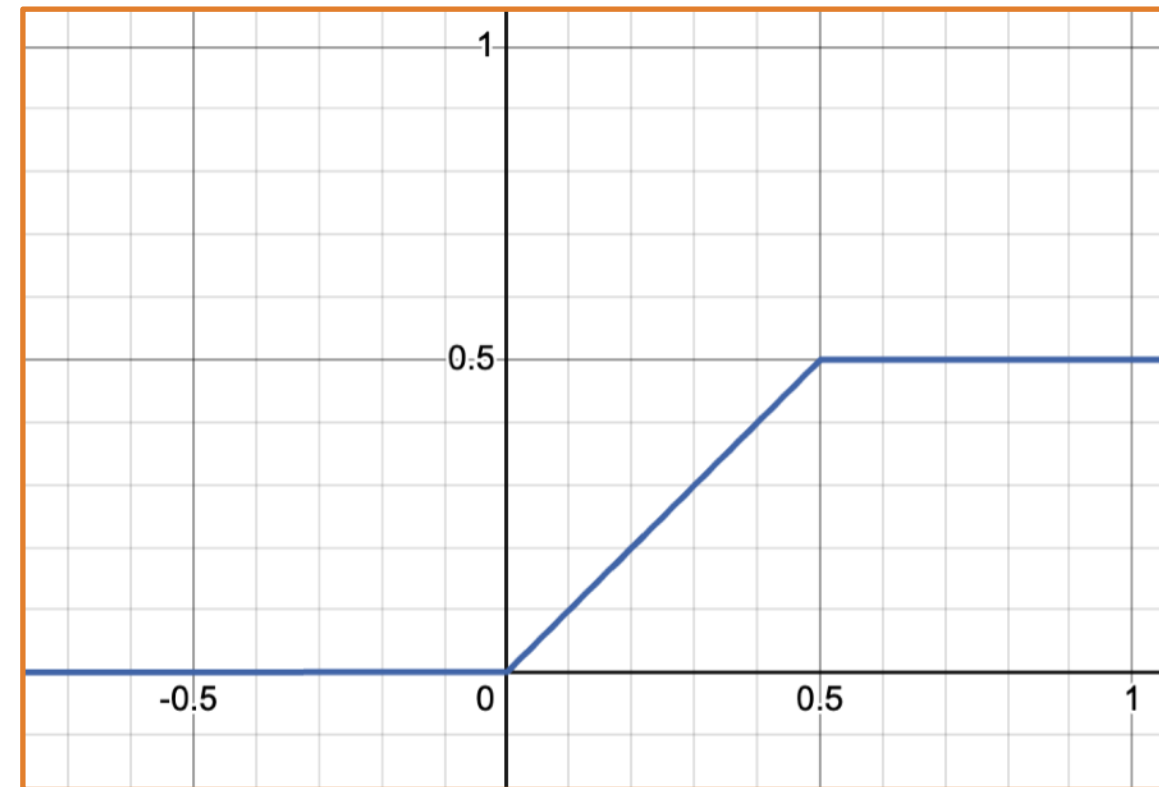
so this above $\wedge$ is $U(x, 0, 1)$

We want a block of height w from $x=0$ to $x=1$.

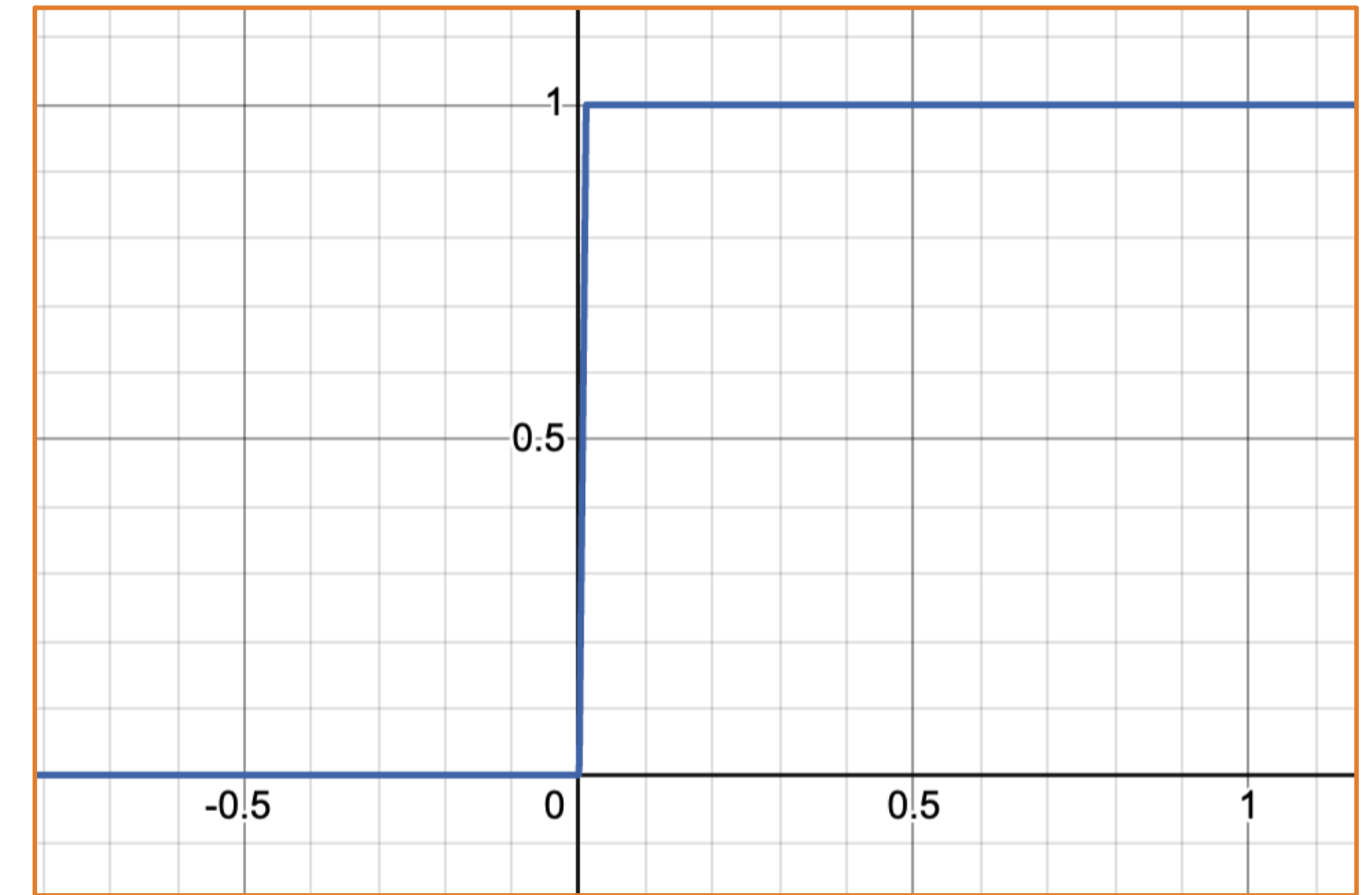
We defined $U(x, t, g) = \text{ReLU}(x - t) - \text{ReLU}(x - t - g)$.



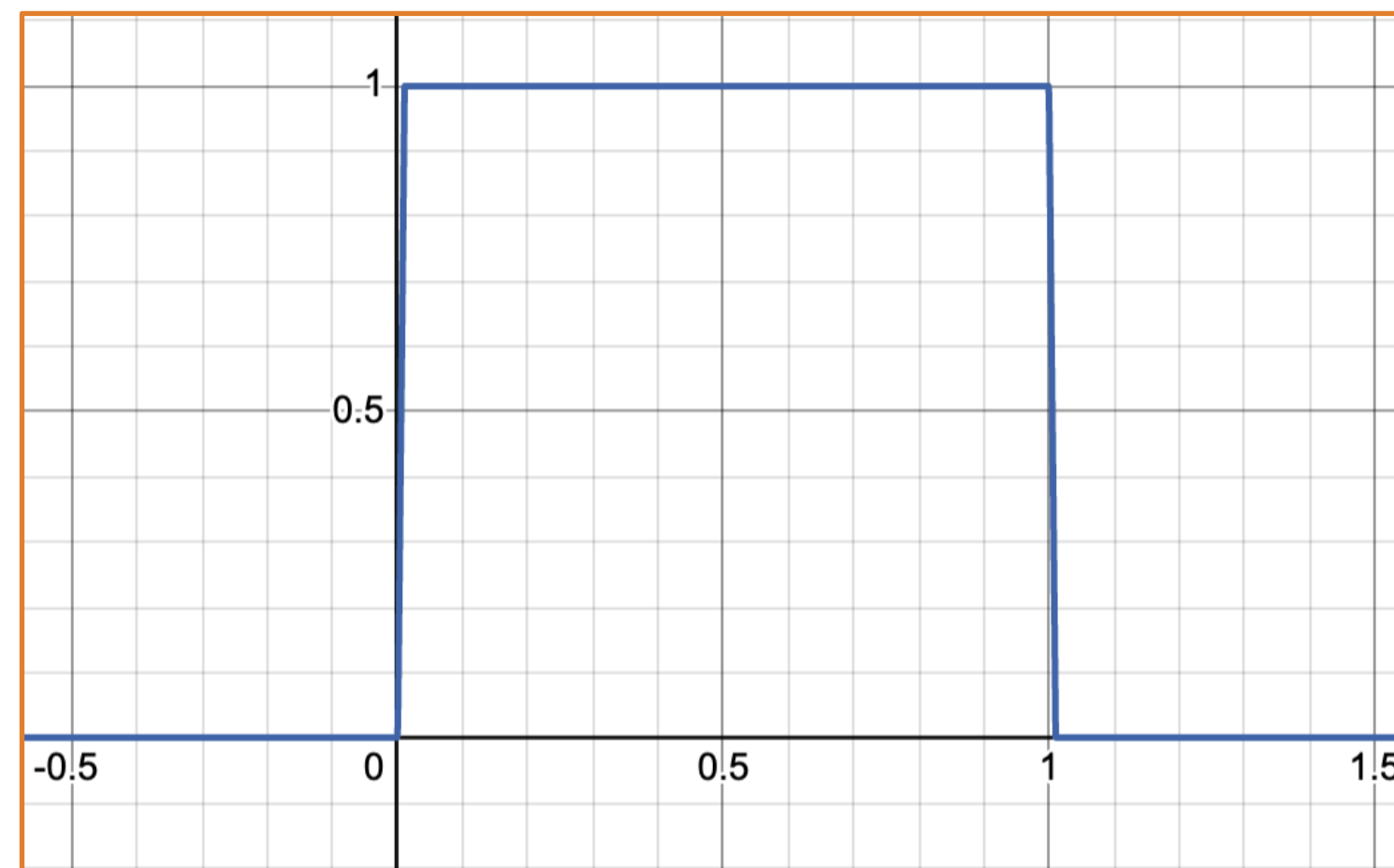
$U(x, 0, 1)$



$U(x, 0, 0.5)$



$100 * U(x, 0, 0.01)$



$\frac{1}{0.01} * [U(x, 0, 0.01) - U(x, 1, 0.01)]$

We want a $\sim$ rectangle of height w from any $\mathbf{x}=\mathbf{u}$ to $\mathbf{x}=\mathbf{v}$.

$$\frac{w}{\gamma} \left[\text{ReLU}(x-u) - \text{ReLU}(x-u-\gamma) - \text{ReLU}(x-v) + \text{ReLU}(x-v-\gamma) \right]$$

with $\gamma > 0$ being a small constant (e.g., $\gamma = 0.001$).

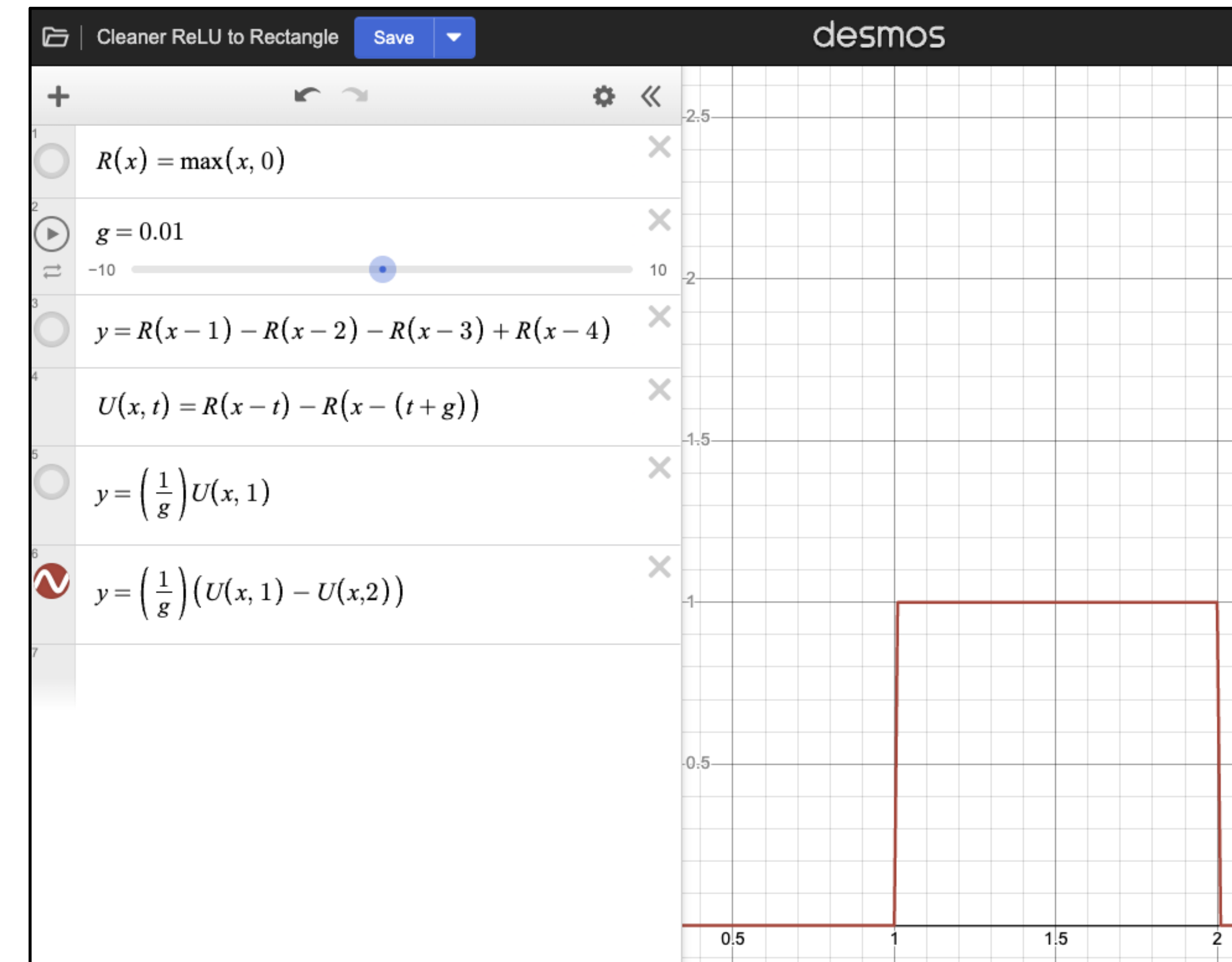
Equivalently, the 2-layer MLP for each “bump” can be written as:

$$\begin{bmatrix} \frac{w}{\gamma} & -\frac{w}{\gamma} & -\frac{w}{\gamma} & \frac{w}{\gamma} \end{bmatrix} \text{ReLU} \left(\begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} x + \begin{bmatrix} -u \\ -(u+\gamma) \\ -v \\ -(v+\gamma) \end{bmatrix} \right)$$

This gives us one rectangle at one position.

We need L / ε of these rectangle.

So just do this L / ε times and sum up the responses.

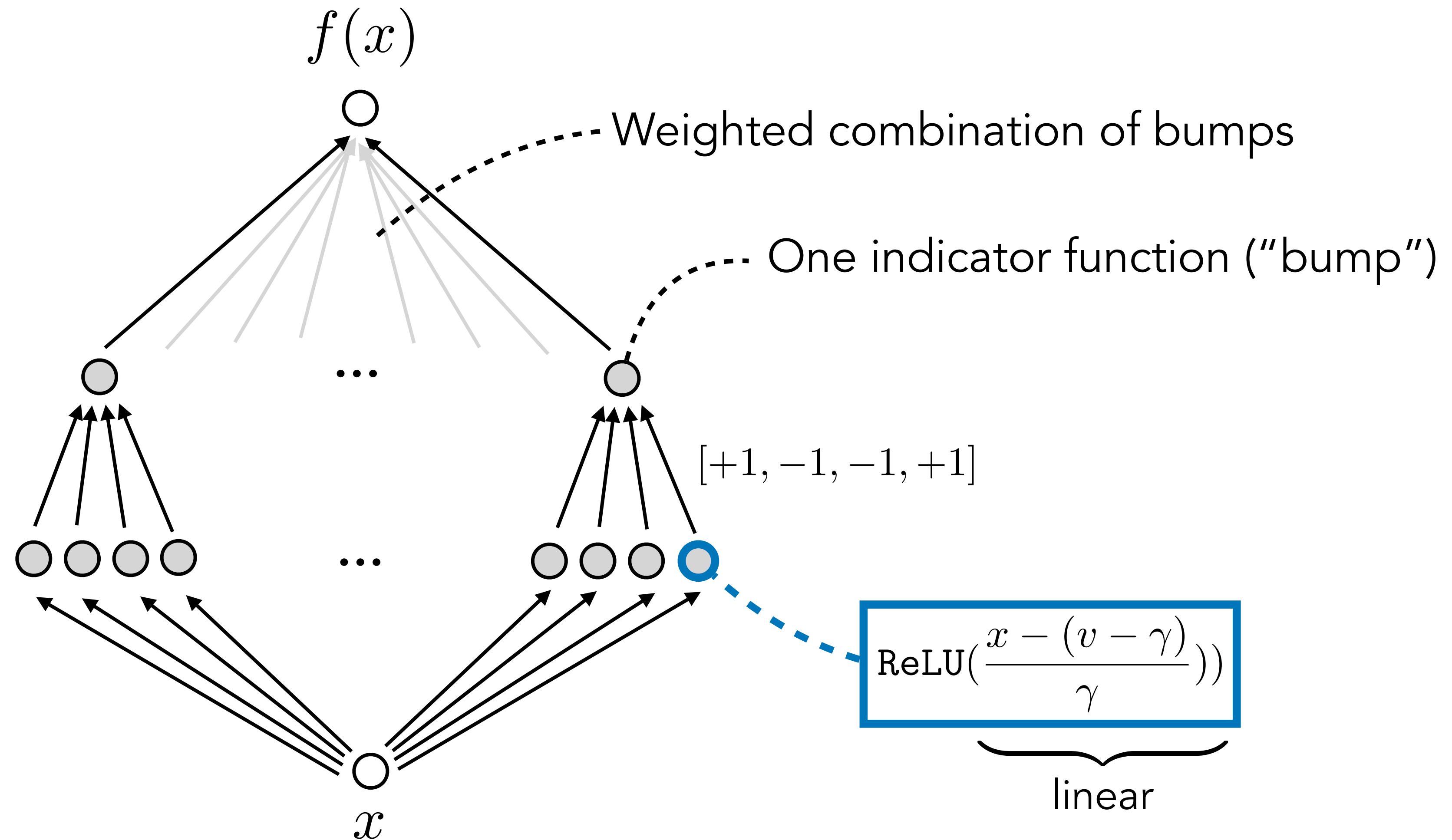


Useful graphing website in 2D and 3D at [desmos.com](https://www.desmos.com) (used for the illustrations)

Approximating a univariate function with an MLP

$$f : \mathbb{R} \rightarrow \mathbb{R}$$

3-Layer ReLU network



Layer 2 and 3 are both strictly linear, hence can collapse into a single linear layer, hence: a 2-layer ReLU net can approx any continuous univariate function arbitrarily well.

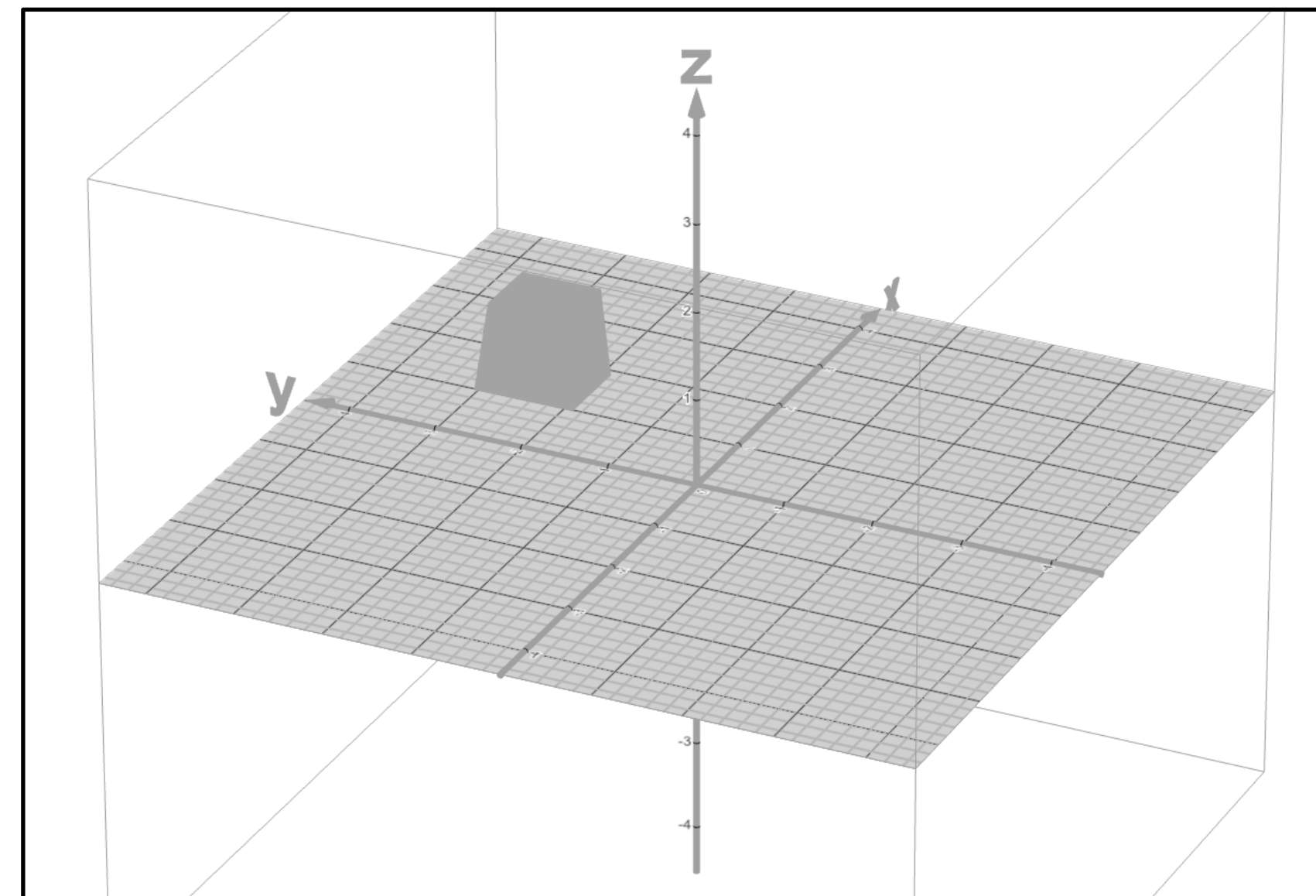
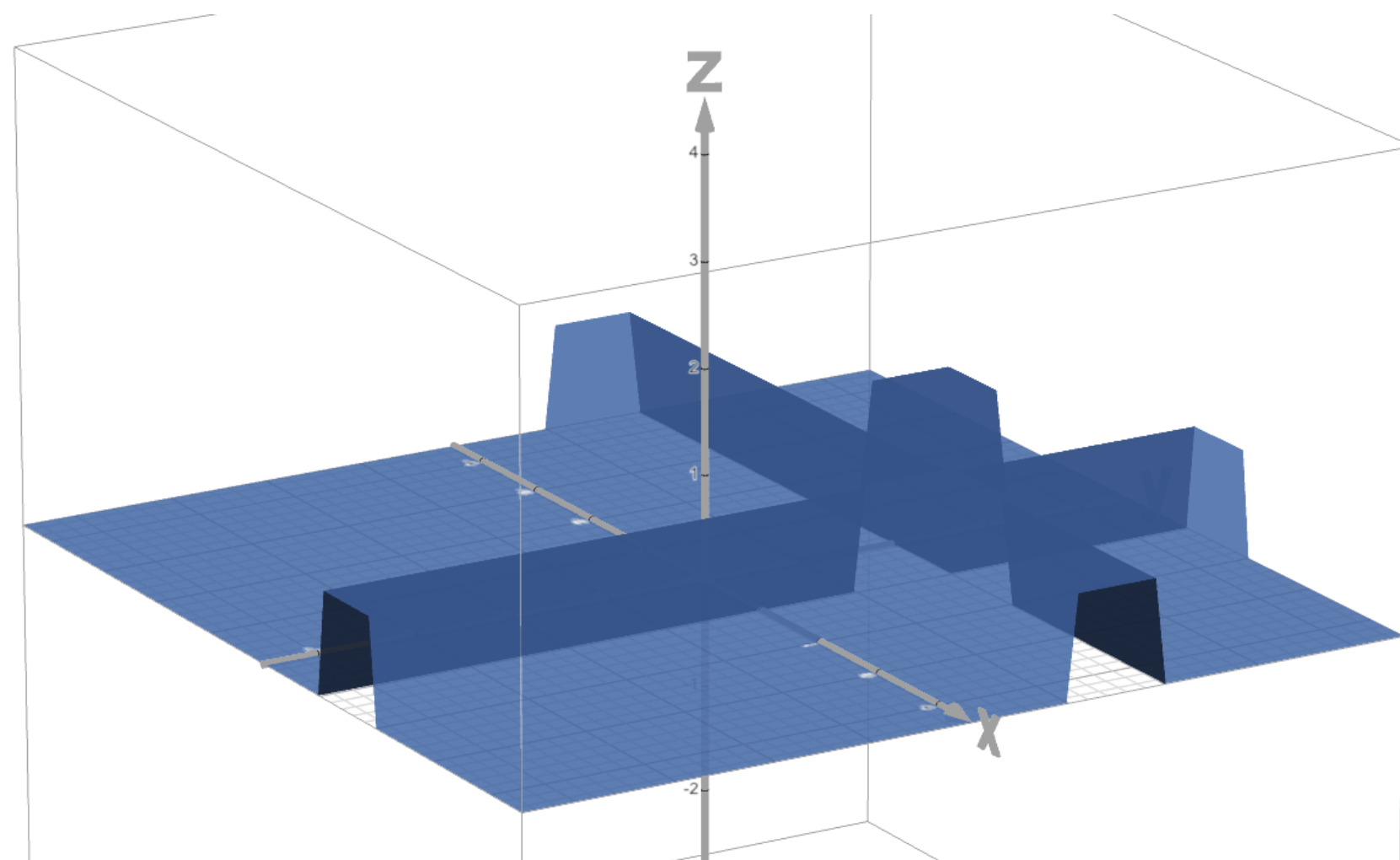
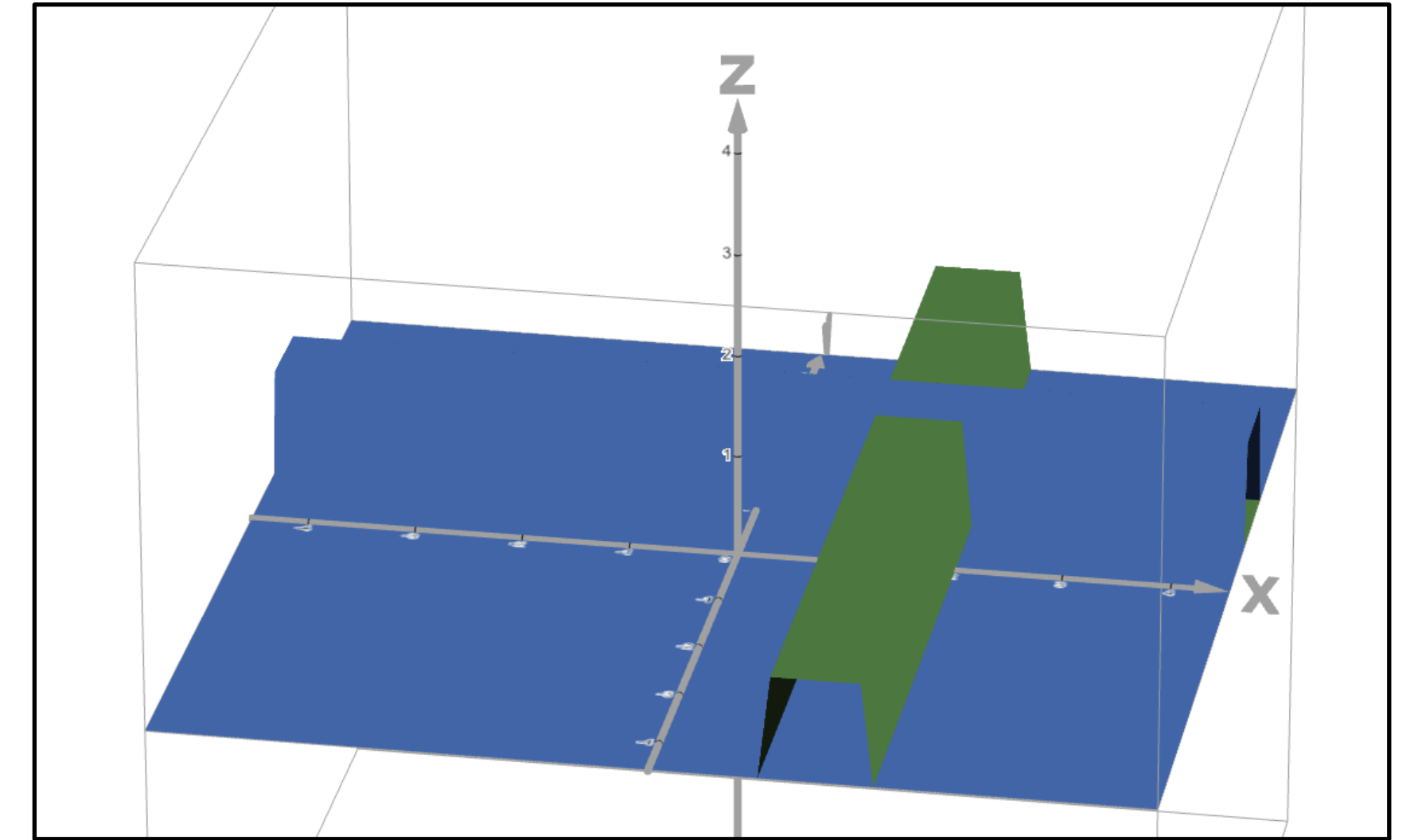
Extending this to multiple dimensions is just addition across dimensions + a threshold at $d-1$, where they overlap.

For each dimension x_i , we'll have the same indicator as before:

$$f_c(x_i; u_i, v_i) = \frac{w}{\gamma} [\text{ReLU}(x_i - u_i) - \dots]$$

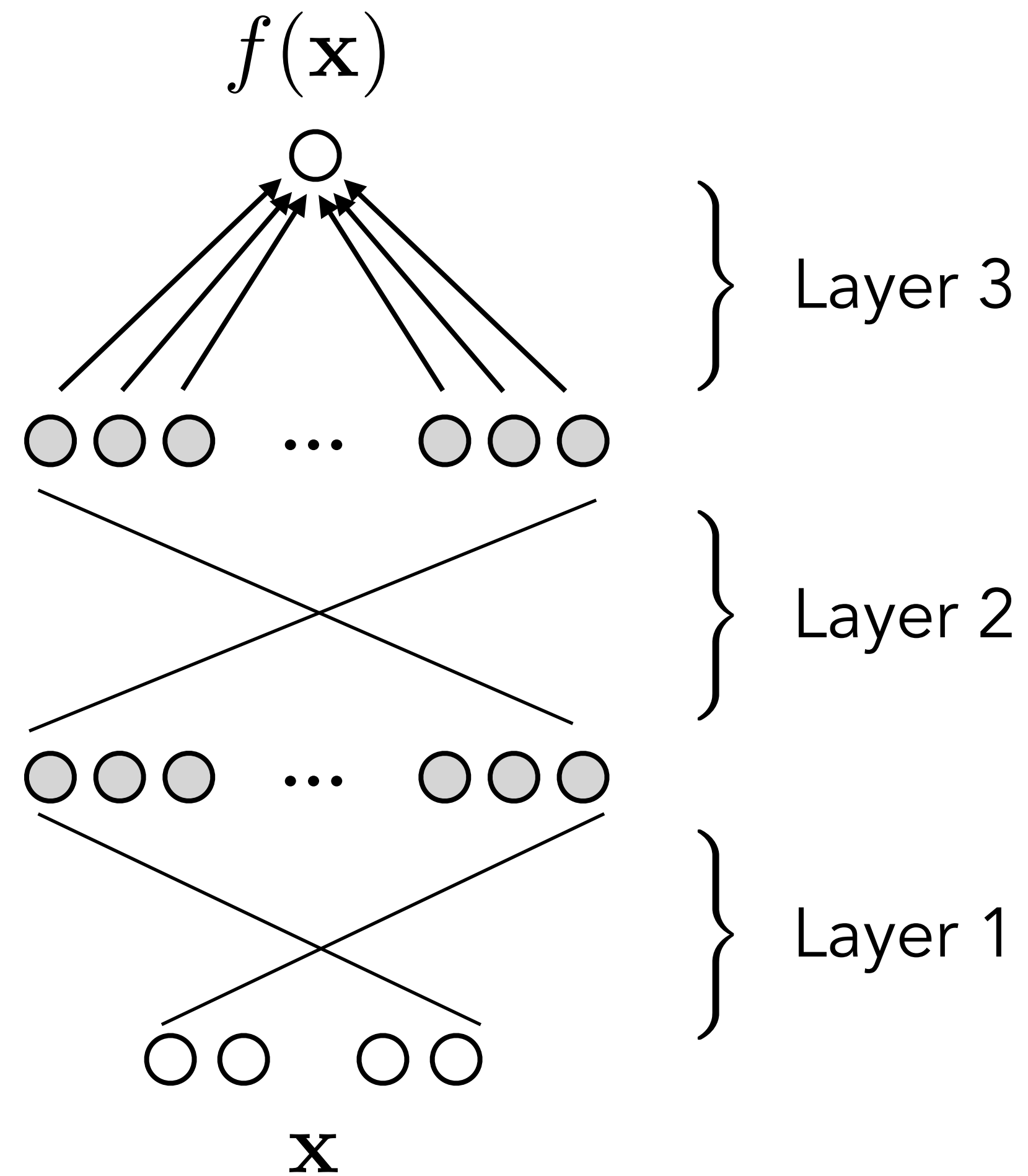
And we'll add them up across dimensions to get our “hyper-rectangle”:

$$h_c(\mathbf{x}; \mathbf{u}, \mathbf{v}) = \text{ReLU} \left(\sum_{i=1}^d f_c(x_i; u_i, v_i) - (d-1)w \right)$$



Approximating a multivariate function with an MLP

$$f : \mathbb{R}^d \rightarrow \mathbb{R}$$



Overall proof structure

Theorem. Let g be an arbitrary L -Lipschitz function and $\varepsilon > 0$. Then there exists a 3-layer ReLU network f with $\Omega((L/\varepsilon)^d)$ units such that

$$\int_{[0,1]^d} |f(\mathbf{x}) - g(\mathbf{x})| d\mathbf{x} \leq 2\varepsilon.$$

1. This is easy to show if we had explicit indicator functions (the if statements), starting in one dimension and generalizing to multiple.

You can understand this Python as:

$$f_{\text{step}}(x) = \sum_j \alpha_j \mathbf{1}[x \in R_j]$$

```
def approx(x): # 5 bins
    if x < 0.2: return 1.02 # ≈ f(0.2)
    elif x < 0.4: return 1.64 # ≈ f(0.4)
    elif x < 0.6: return 0.86 # ≈ f(0.6)
    elif x < 0.8: return 0.48 # ≈ f(0.8)
    else: return 0.64 # ≈ f(1.0)
```

2. With some ReLU compositions, it's easy to approximate each indicator explicit bump, if our gamma is small enough.
3. We just then need to add up all the bumps, with one final layer.

Some limitations of the statement we proved.

- We're talking about width $(L / e)^d$, i.e. exponential in the number of input features!
Think about what this means when you have "just" 1000 features.
 - We did NOT prove that it's **required** for DNNs. Only that our proof needs so many!
- We needed very large weights w/γ . Large weights are often a sign that something is going wrong, e.g. here, it's an explicit lookup-table.
- Limitation of the proof sketch: We didn't account for the approximation error from the fact that γ is not actually zero. This costs us in the statement: approximation only on average (the integral), not pointwise.

Further reading and improvements

- **Universal approximation (2-layer, pointwise)** (Hornik, Stinchcombe, White 1989, Leshno et al 1993):

Suppose we use a continuous activation function that is not a polynomial. Then for any continuous function $g : \mathbb{R}^d \rightarrow \mathbb{R}$ and any ϵ , there exists a 2-layer network f with bias terms such that $|f(x) - g(x)| \leq \epsilon$ for all $x \in [0,1]^d$

- **Barron's theorem:** smooth functions can be approximated with fewer neurons.

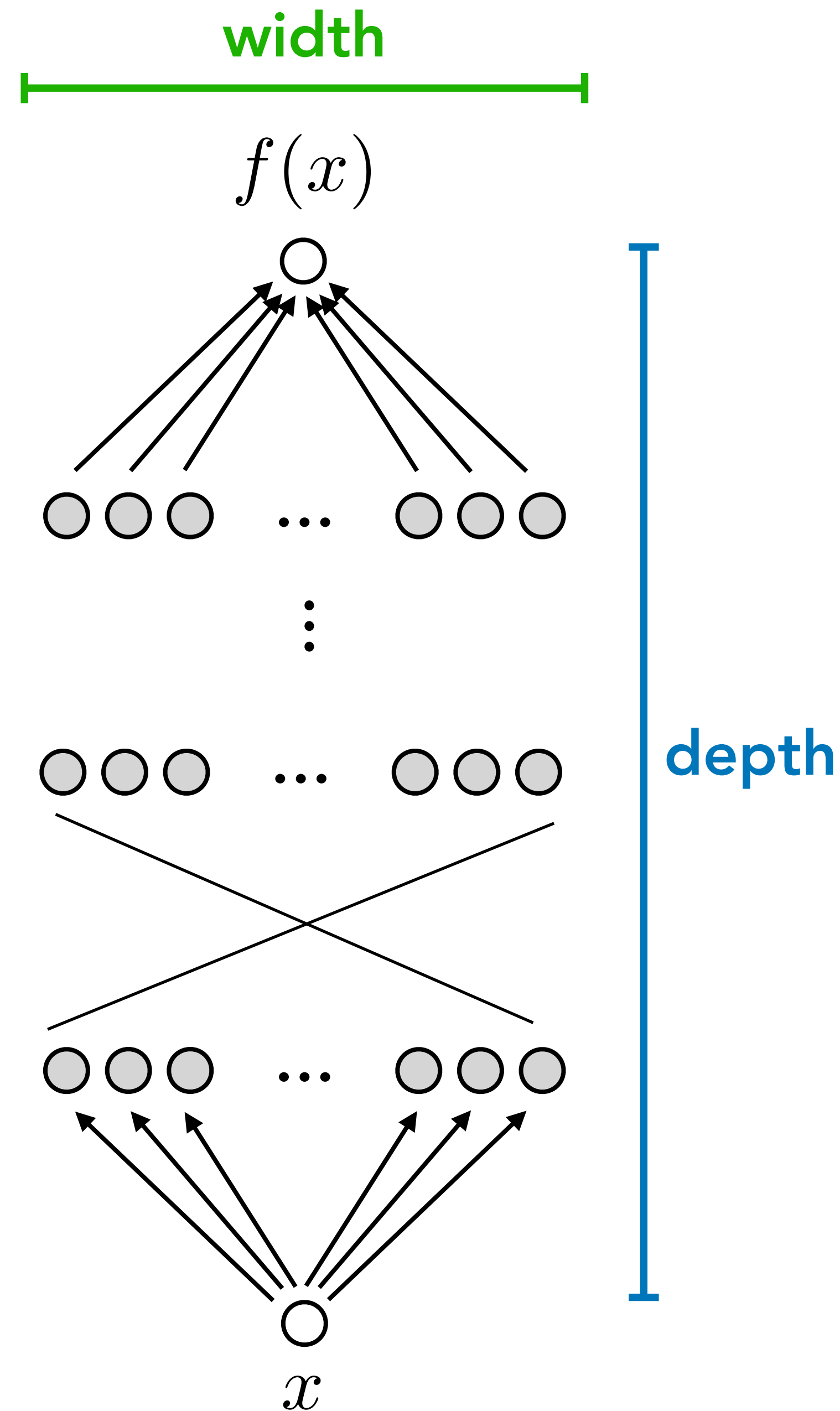
Should you scale the width or depth
of your neural network?



Depth vs width

$$f : \mathbb{R} \rightarrow \mathbb{R}$$

Main question: Is it better to add width or to add depth?



Why might you want to focus on scaling width?

- We just learned that even a 2-layer MLP is a “universal approximator”... if it’s wide enough!
- Width is inherently parallelizable. Depth is sequential.
- Width is easier to train. Depth can lead to gradient problems.

So... scaling width is obviously better?

Why might you want to focus on scaling depth?

- It works better in practice!
- It makes sense intuitively for feature processing?
- And.. “depth separation” results!

It's possible to describe classes of functions that

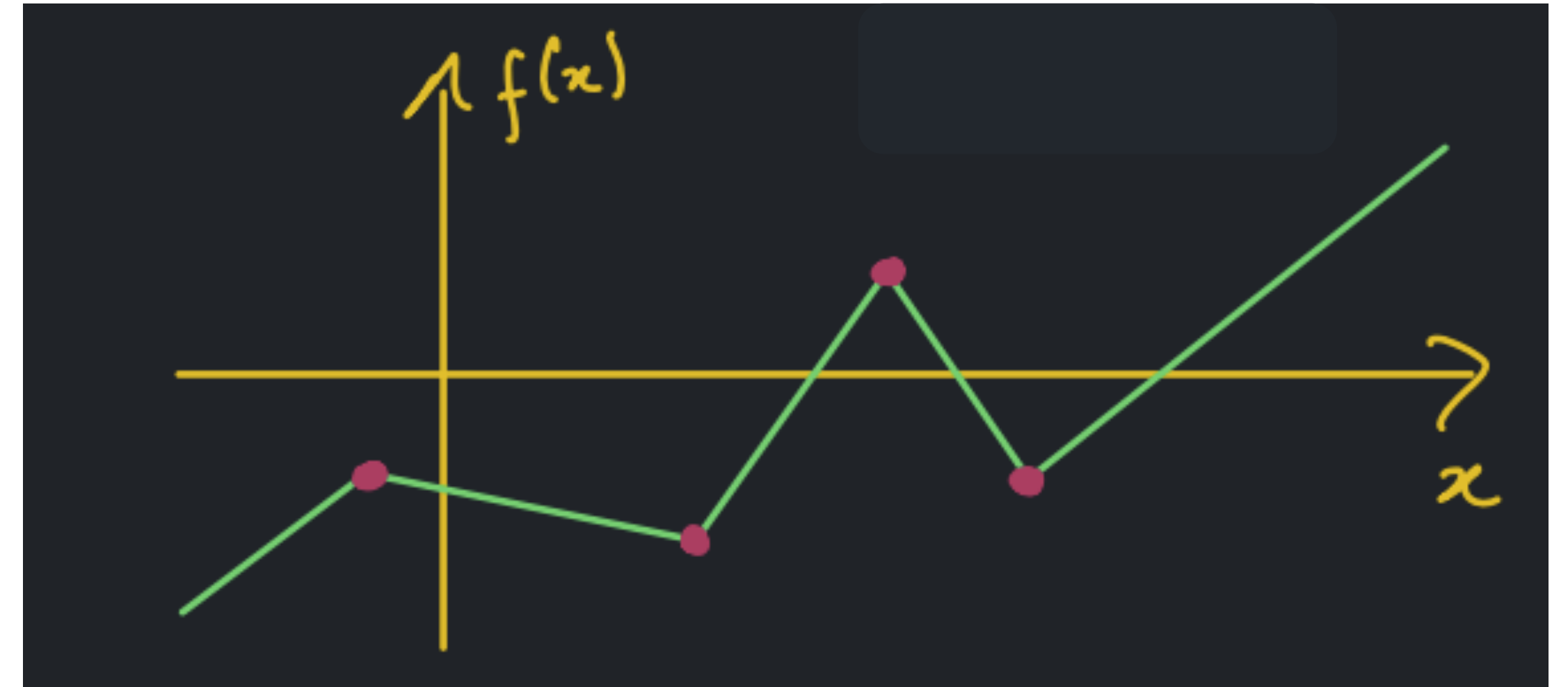
- don't require a lot of neurons if you stack them deeply
- but require exponentially more units to fit with shallow networks!

We'll describe two: Piecewise linear functions & Sawtooth functions.

Piecewise linear ("PWL") functions

Define a kink to be a place where the gradient changes.

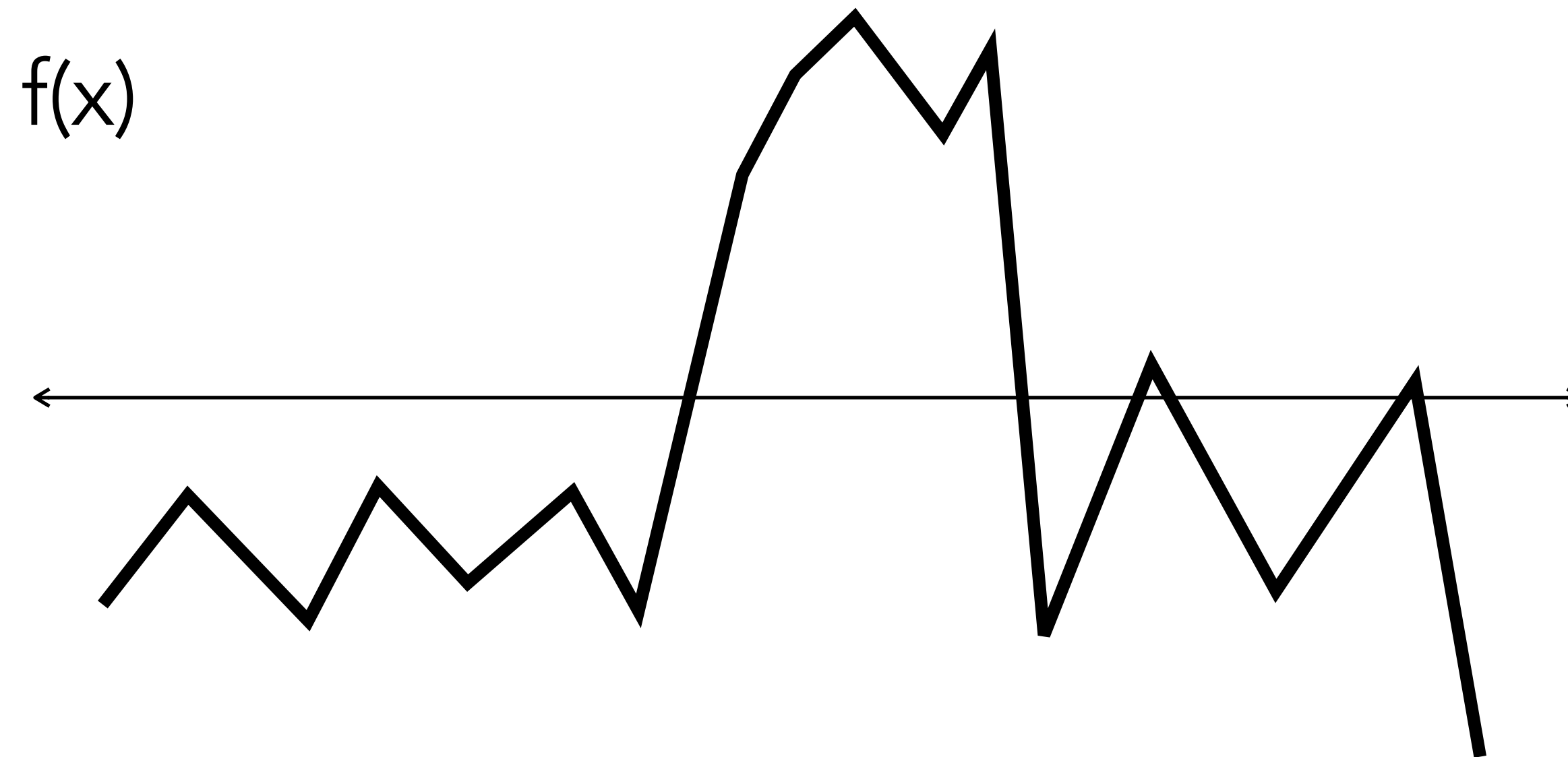
Here, we have 4 kinks.



Claim: ReLU networks are PWL. Why?

1. Linear layers are PWL. /
2. ReLU is PWL. ---/
3. f, g both PWL $\Rightarrow f \circ g$ is PWL

Depth vs width for PWL functions

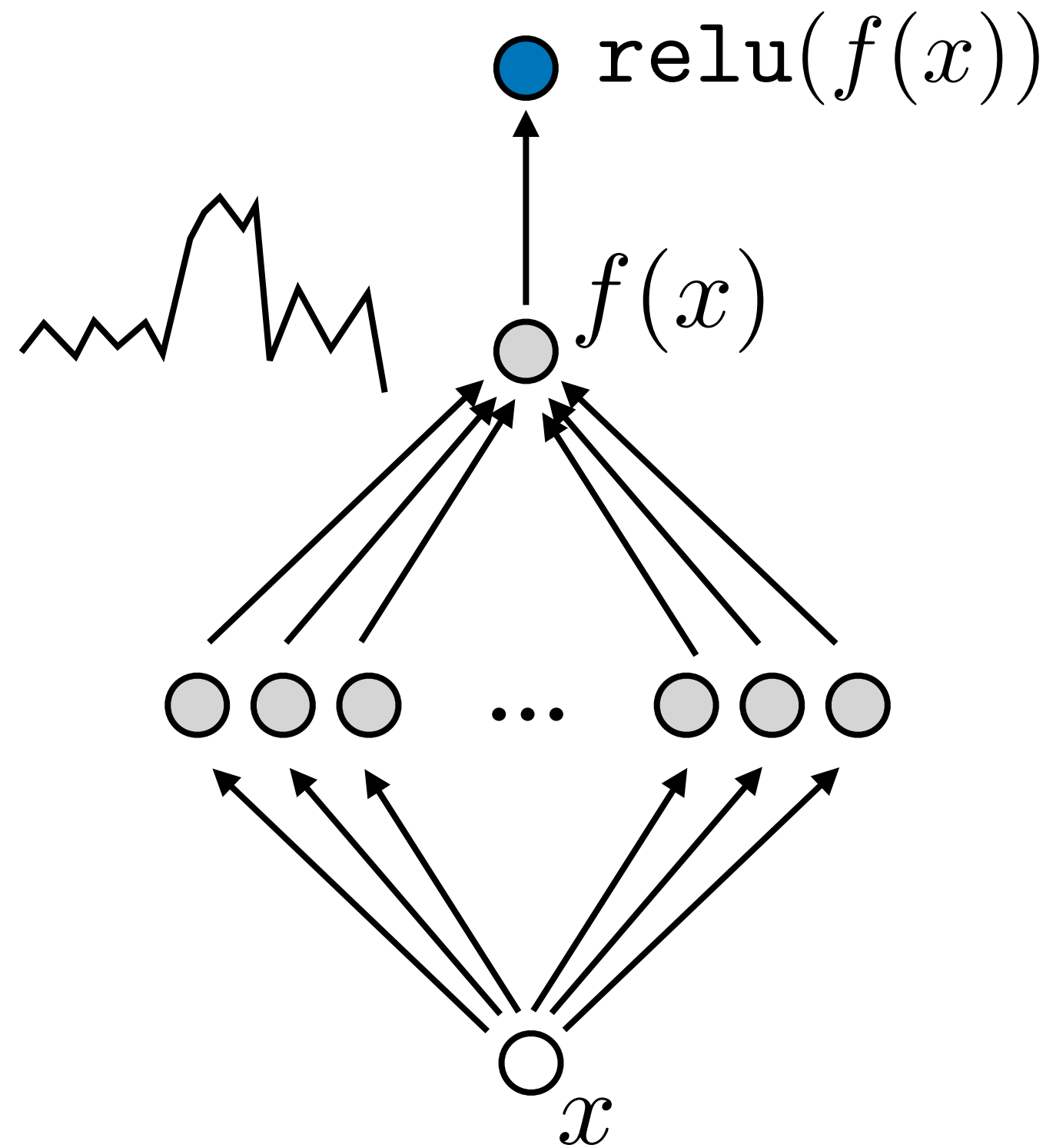


The number of linear pieces increases at most:

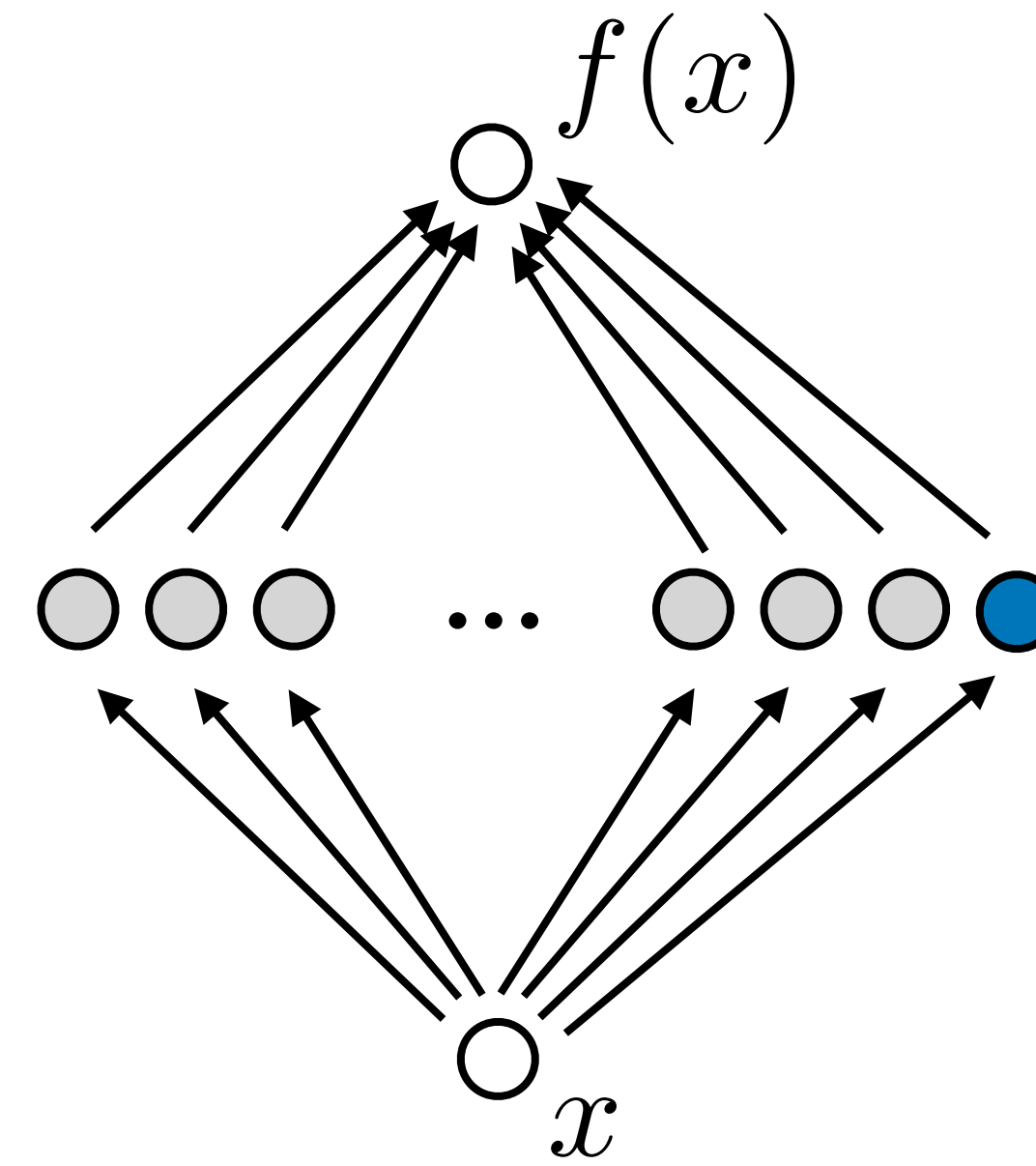
- polynomially with width
- exponentially with depth

Depth vs width

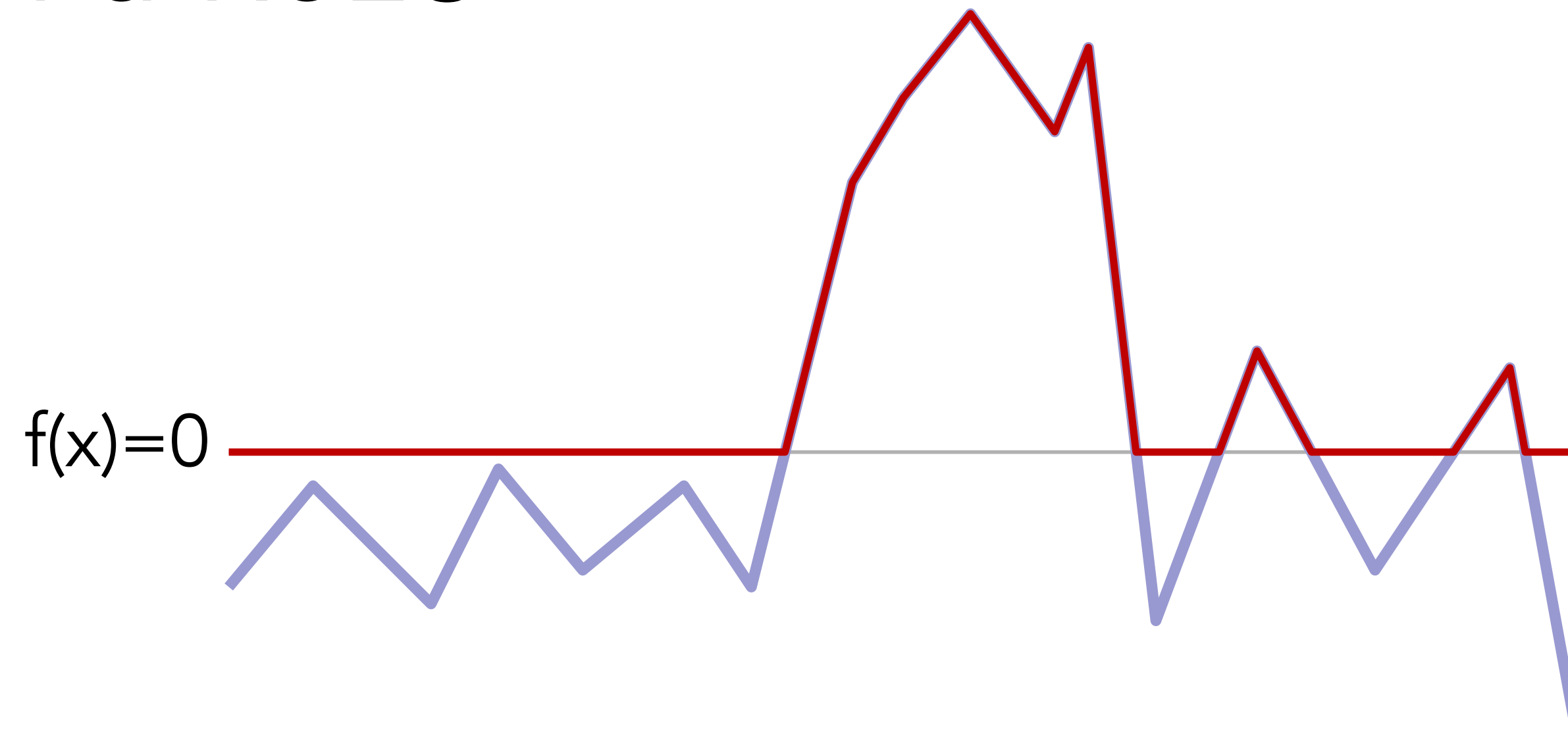
Let's compare adding **one neuron** in depth, or **one neuron** in width.



VS



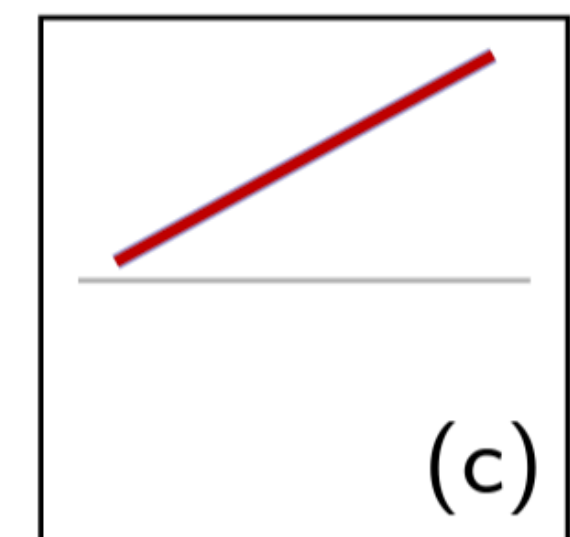
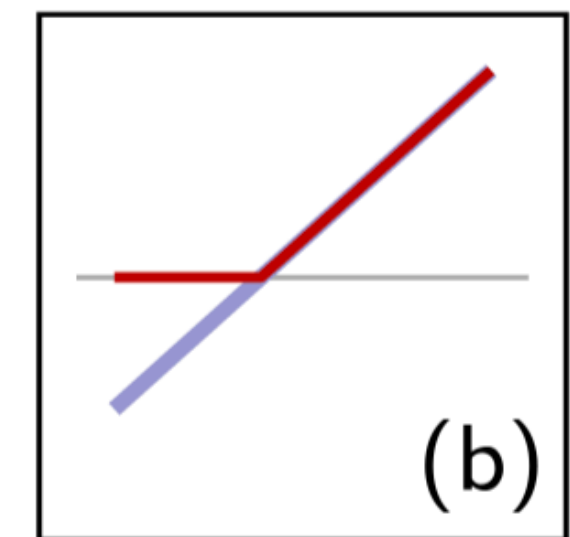
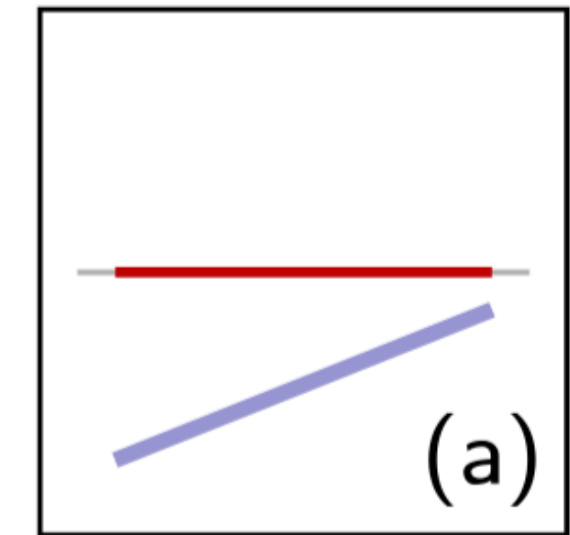
Effect of a ReLU



ReLU can potentially split each linear piece.

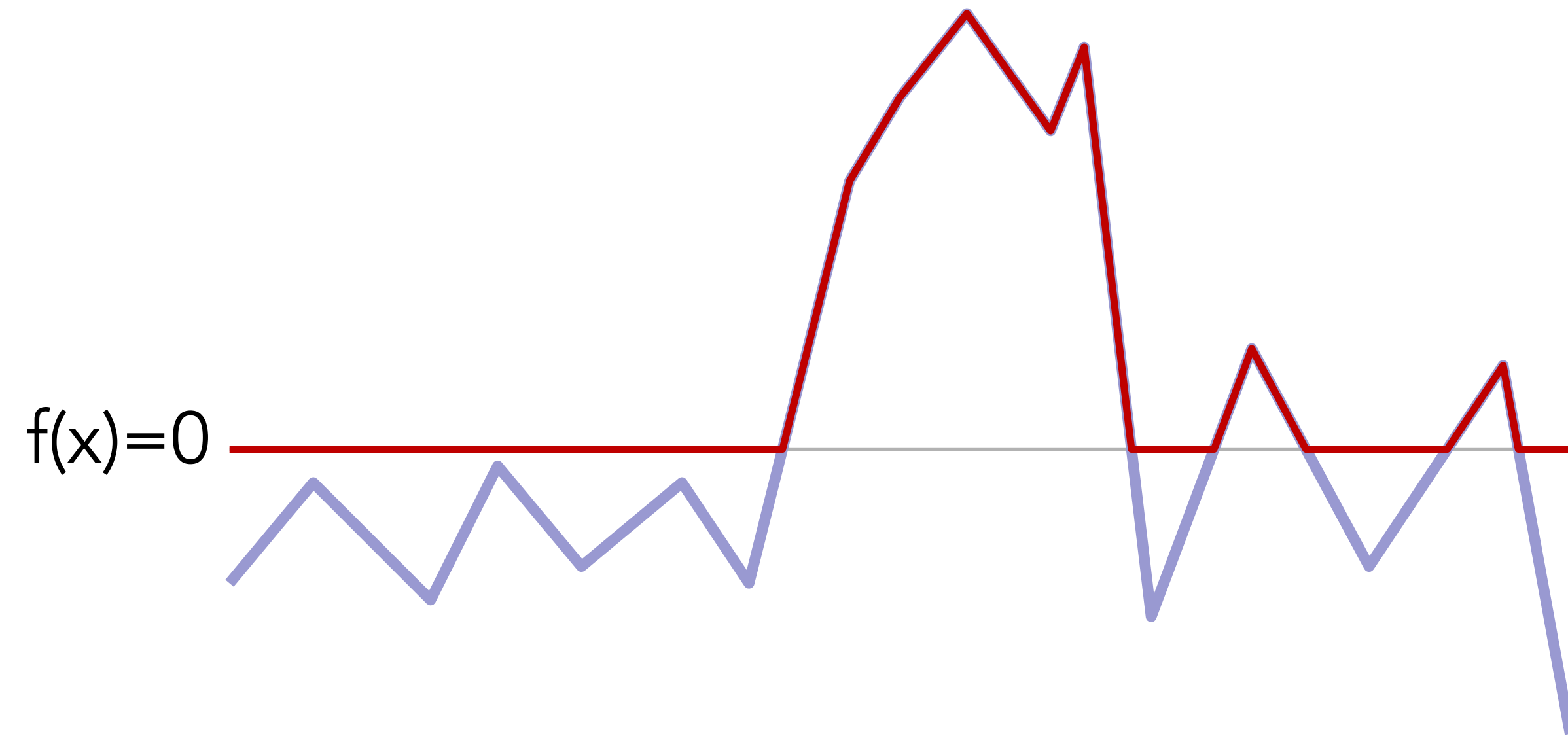
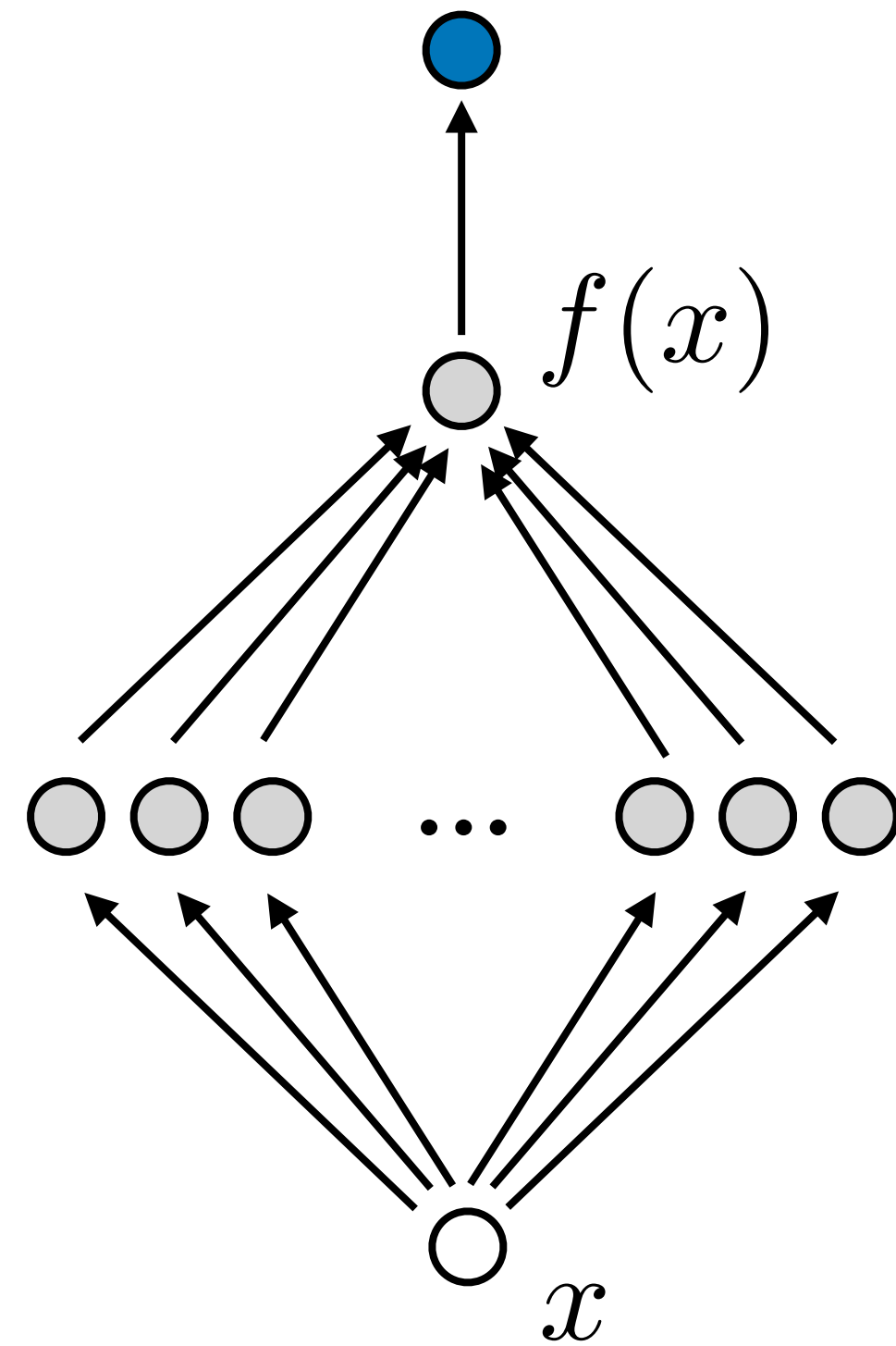
If the input has N such pieces, the output can have up to $2N$.

Each additional layer in depth can double the number of linear pieces.



$f(x)$ — blue line
 $\text{ReLU}(f(x))$ — red line

Adding one neuron in depth (one more layer)

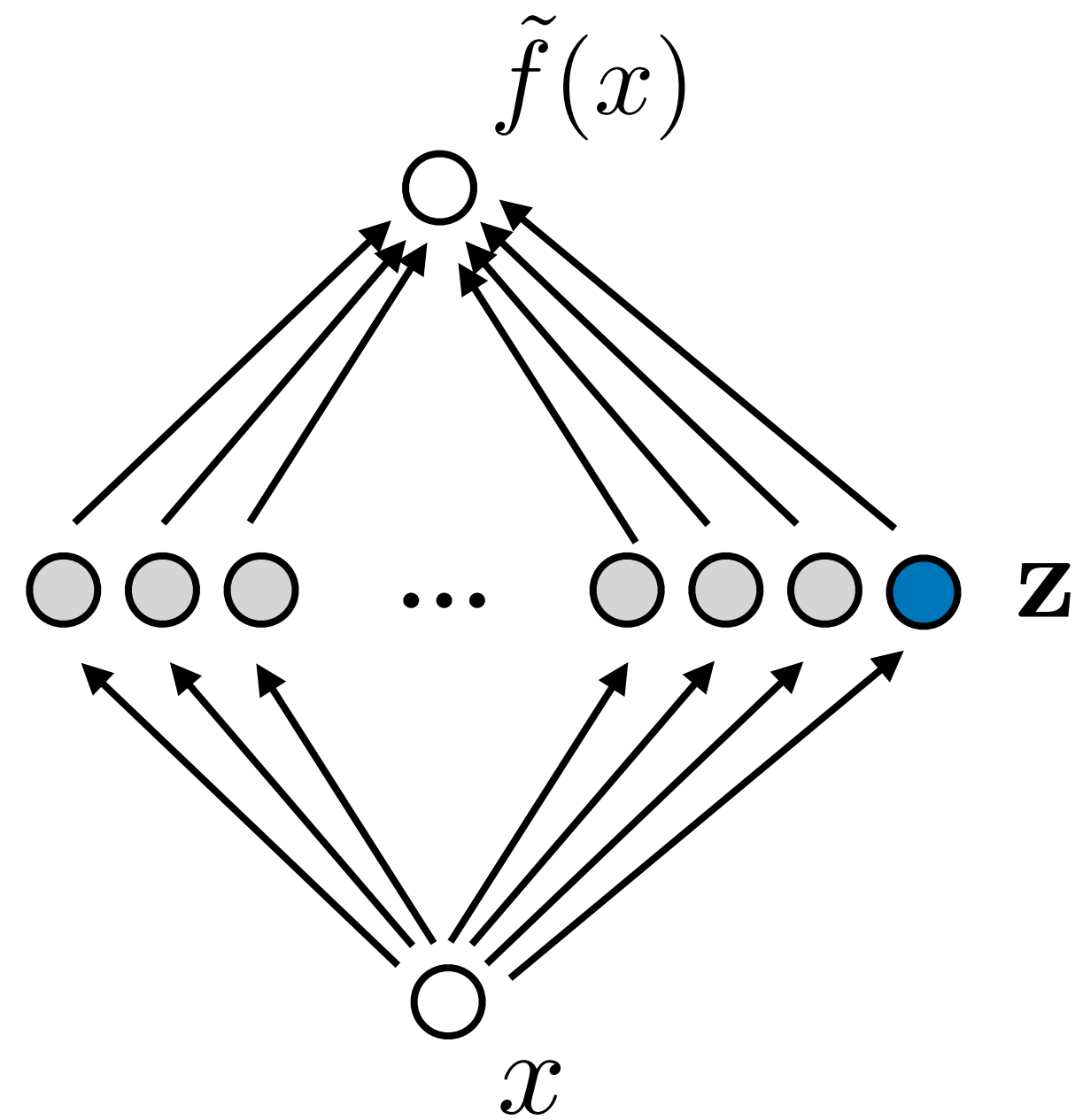


Let $\kappa(x)$ be the number of linear pieces in x .

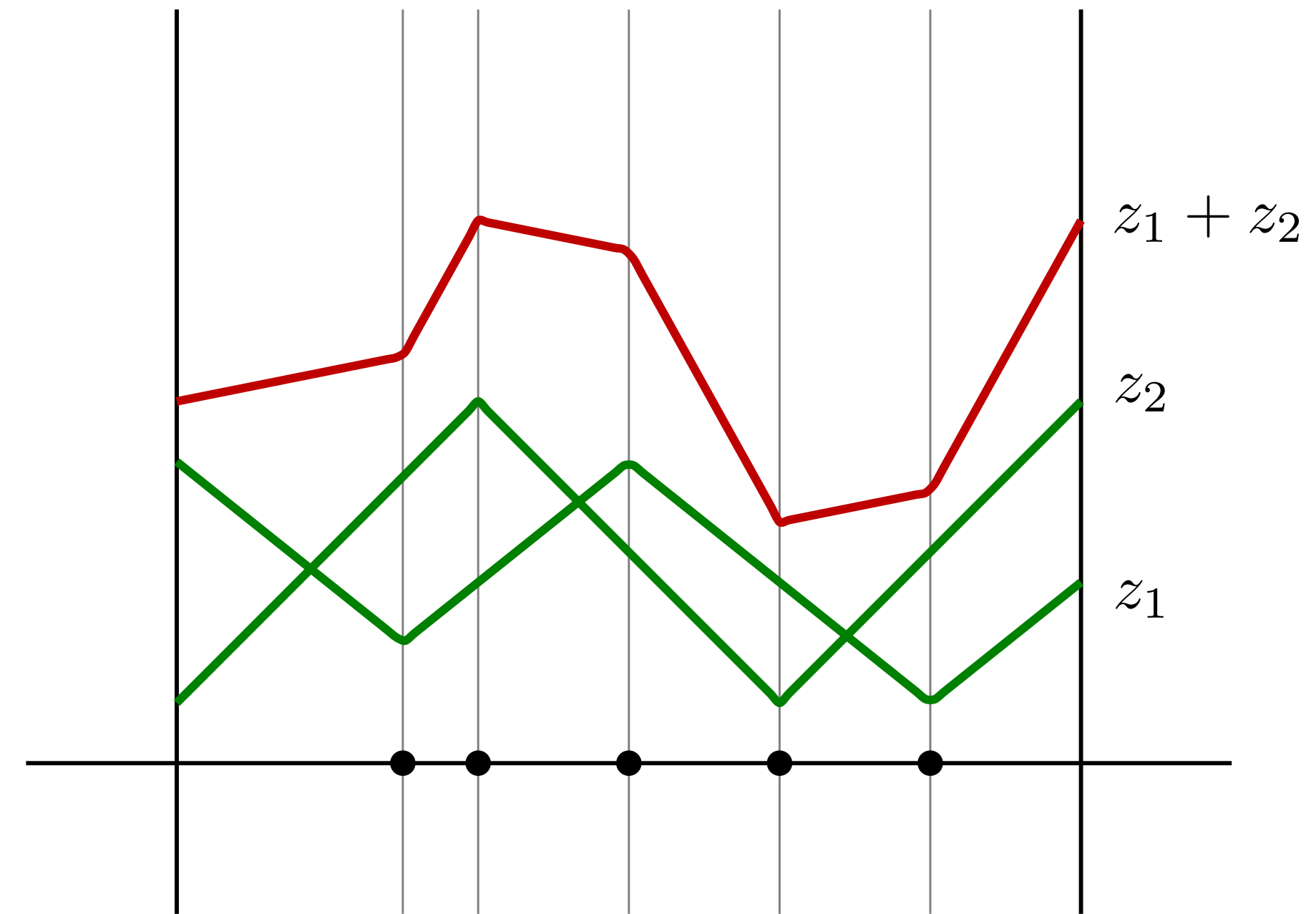
$$\kappa(\text{relu}(f(x))) \leq 2\kappa(f(x))$$

Max number of linear pieces is
exponential in depth

Adding one neuron in width



Max number of linear pieces is
linear in width

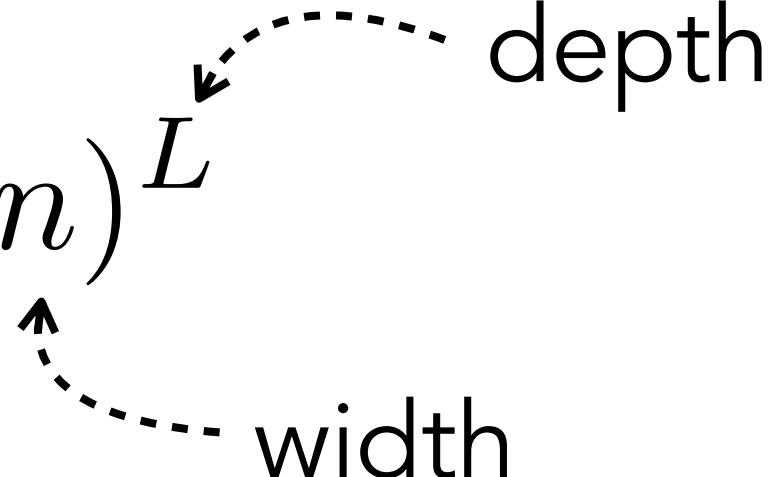


$$\kappa(\tilde{f}(x)) \leq \sum_{i=1}^{n+1} \kappa(z_i)$$

$$\kappa(\tilde{f}(x)) \leq \underbrace{(n+1)}_{\text{width}} K \quad K = \max_i \kappa(z_i)$$

Depth vs width: summary

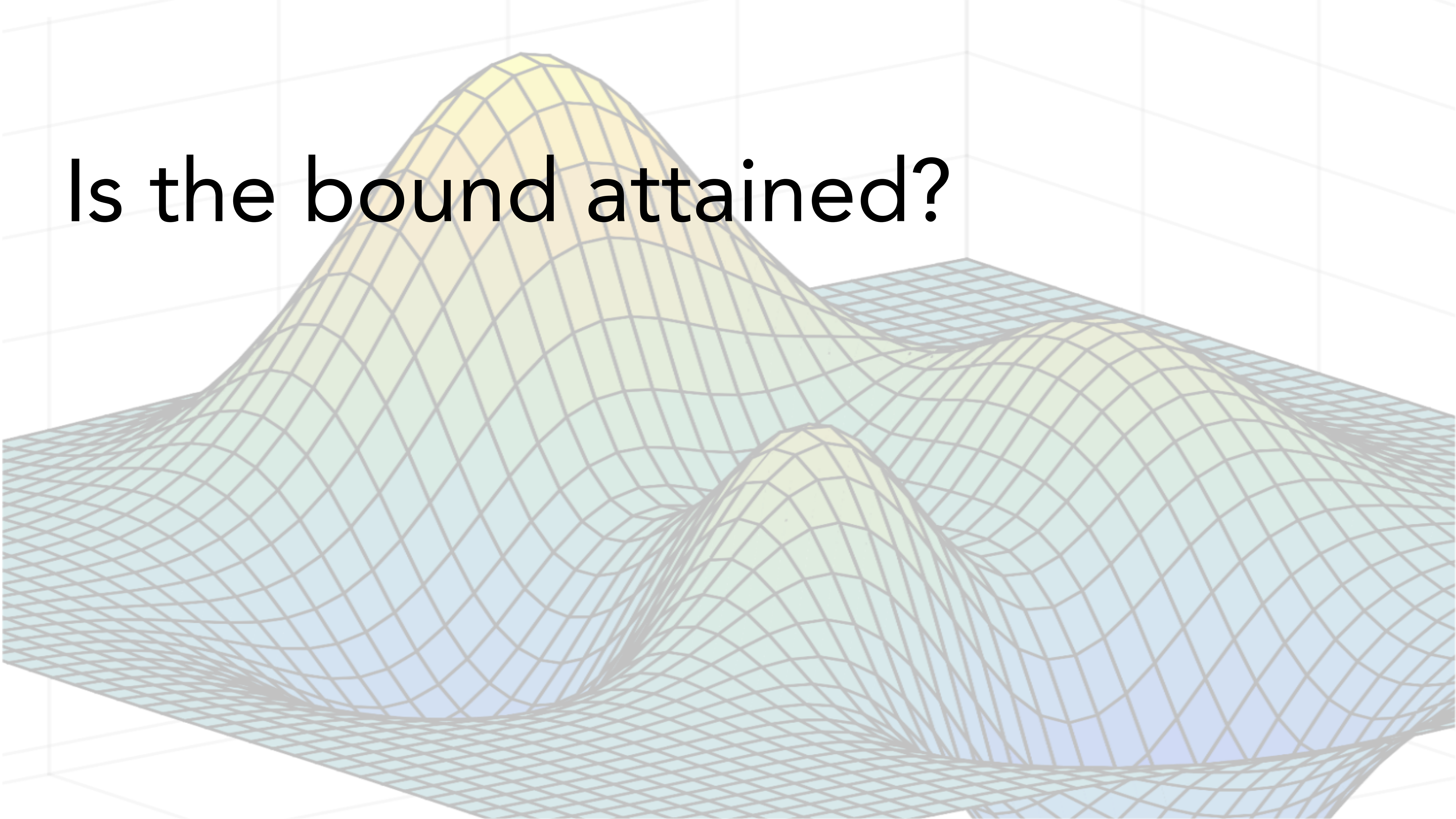
Number of linear pieces increases up to exponentially in depth, but only up to polynomially in width:

$$\kappa(f(x)) \leq (2n)^L$$


For full derivation, see: <https://fleuret.org/dlc/materials/dlc-handout-6-1-benefits-of-depth.pdf>

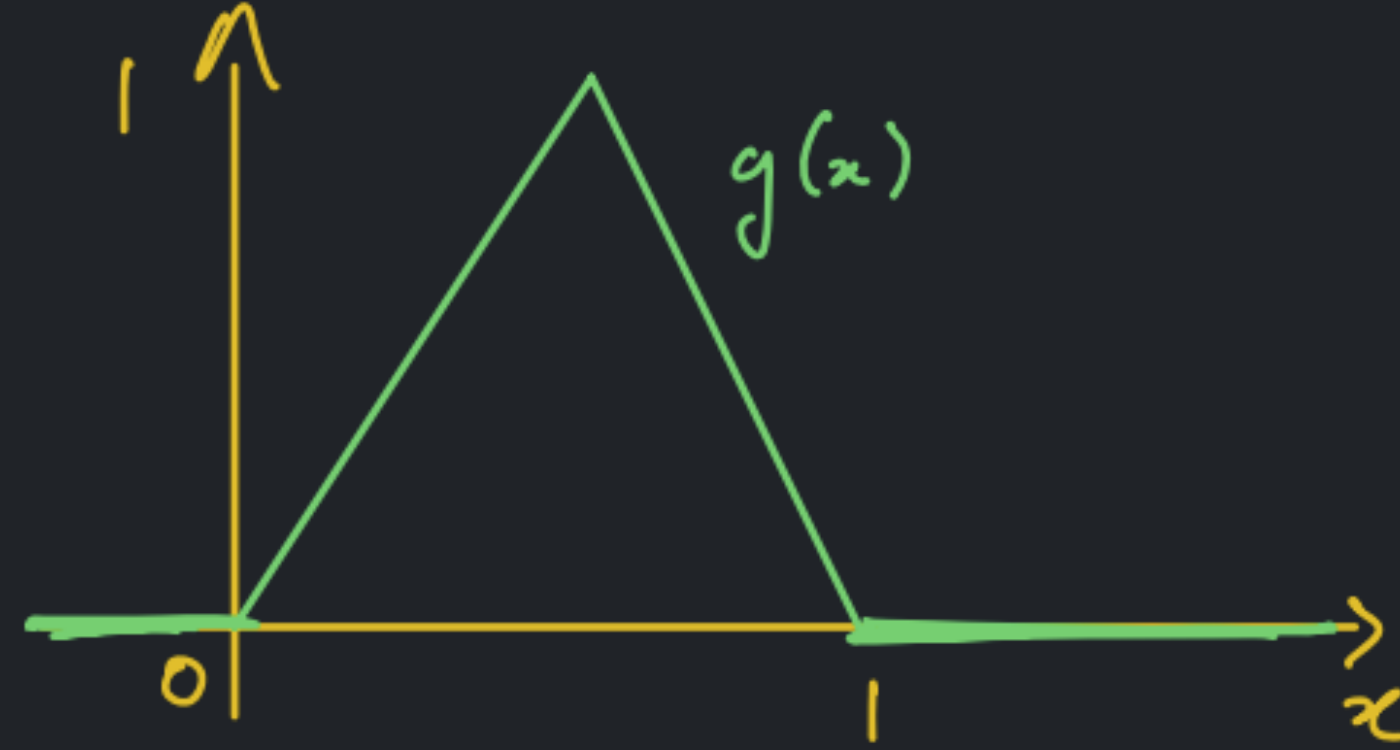
What are the implications for approximating wiggly functions?

Is the bound attained?

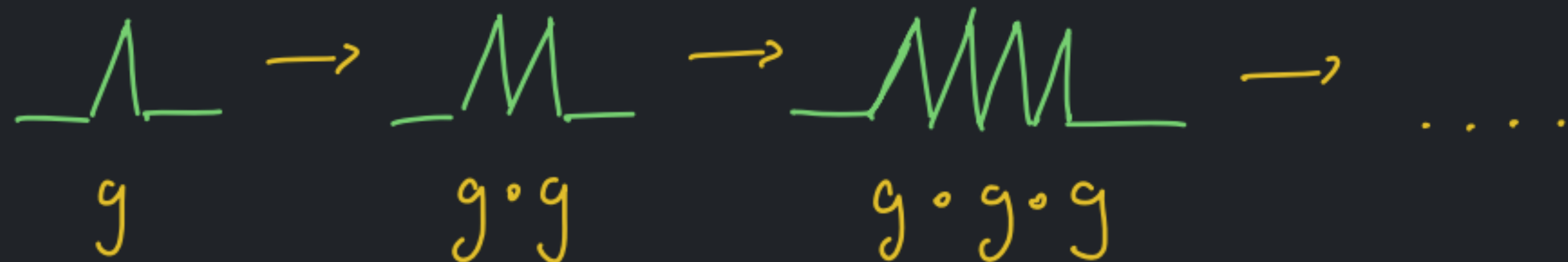


Answer: Yes, with the sawtooth function.

$$g(x) = \text{relu}(2\text{relu}(x) - 4\text{relu}(x - \frac{1}{2}))$$



Each time we "iterate" g — i.e. compose with self — we double the number of linear regions



Depth separation: sawtooth function

Theorem (Telgarsky 2015,2016): For any $L > 1$ there exists a function $g(x)$ such that $g(x)$ can be expressed by a (deep) ReLU network with $3L^2 + 6$ units and $2L^2 + 4$ layers, but any (shallow) ReLU network with L layers and $\leq 2^L$ nodes cannot approximate it to error $\|f - g\|_1 < 1/32$.

Note: You can approximate it with a 2-layer network, but need exponentially more units

Depth Separation for the sawtooth function.

$g \circ g \circ \dots \circ g$ (500 times) has $2^{500} - 1$ kinks

It has 1000 layers, with width 2.

To get the same number of kinks, a 3-layer network would need width n in...

$$\text{Kinks} \leq (2n)^L$$

$$\Rightarrow n \geq \frac{1}{2} \text{Kinks}^{(1/L)}$$

$$= \frac{1}{2} (2^{500} - 1)^{1/3}$$

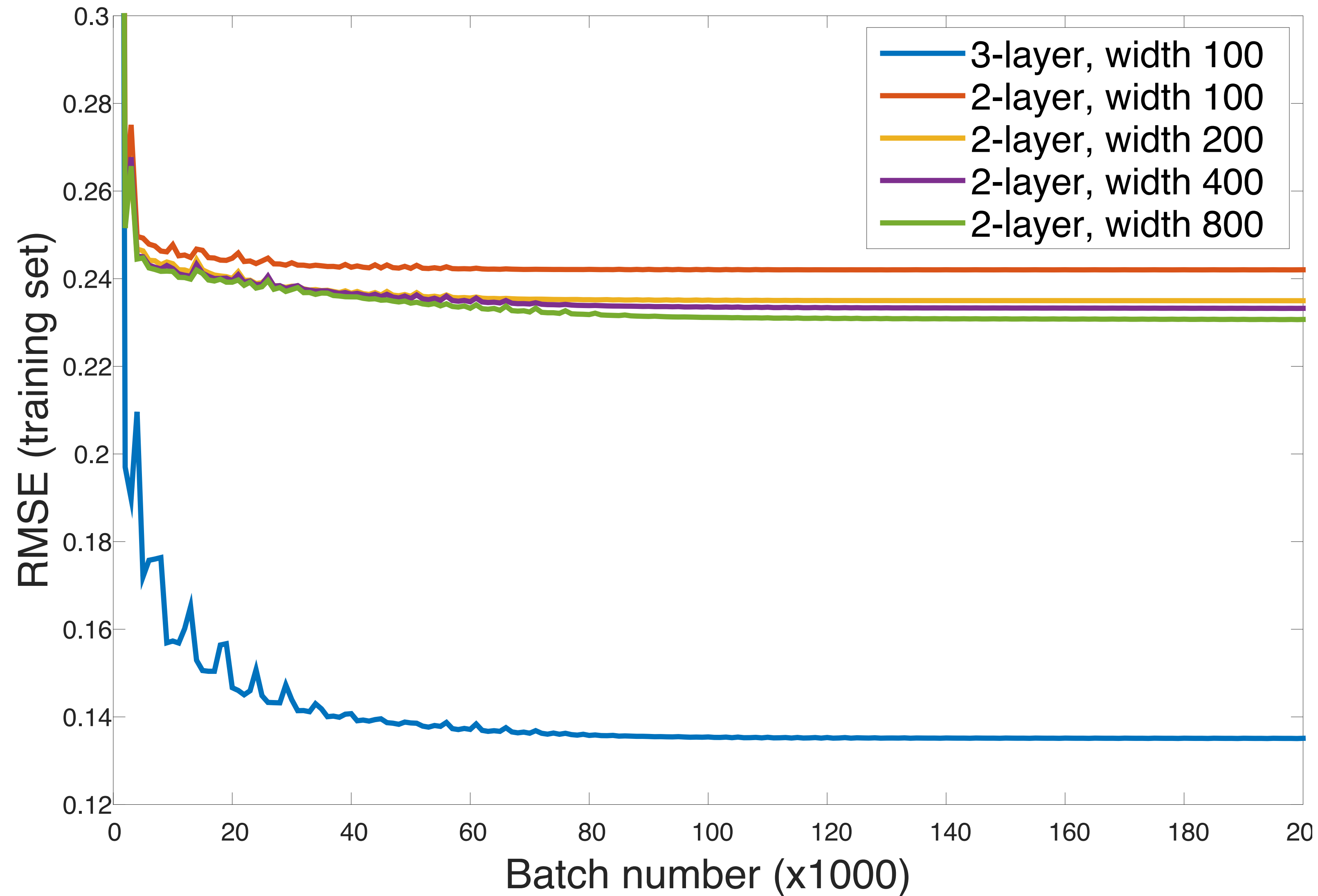
$$= 7 \times 10^{49} \text{ units}$$

Depth separation: more examples

- Indicator of the unit ball in $\mathbb{R}^d$ can be approximated to accuracy ϵ using a 3-layer ReLU net with $O(d/\epsilon)$ neurons. But it cannot be approximated to an accuracy higher than $O(1/d^4)$ using a 2-layer network, unless its width is exponential in d .

$$g(x) = \mathbf{1}[\|x\| \leq 1]$$

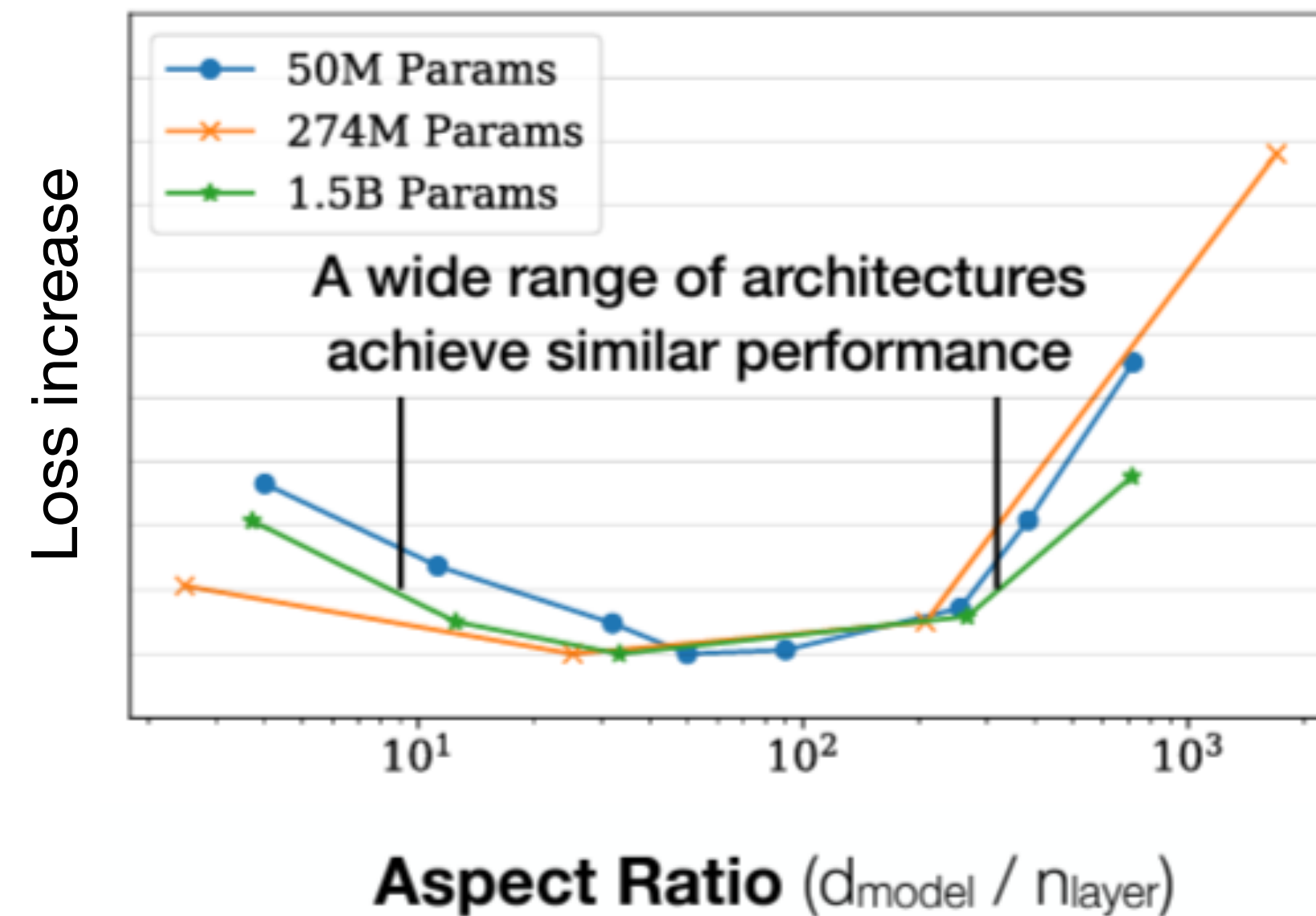
Unit ball experiment



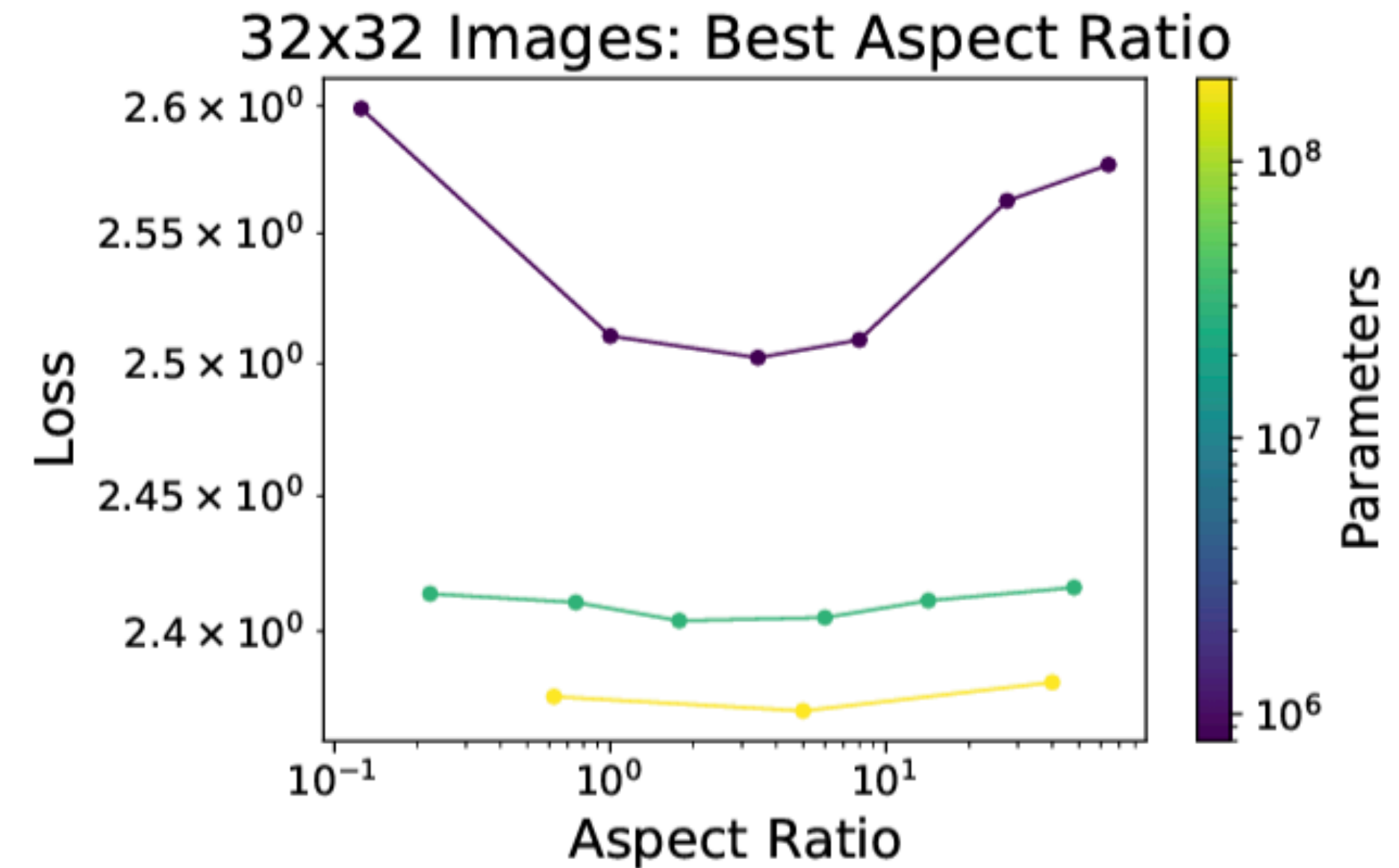
Depth vs width in practice

Main question: Is it better to add width or to add depth?

Ongoing science!



[Kaplan*, McCandlish*, et al. 2020]

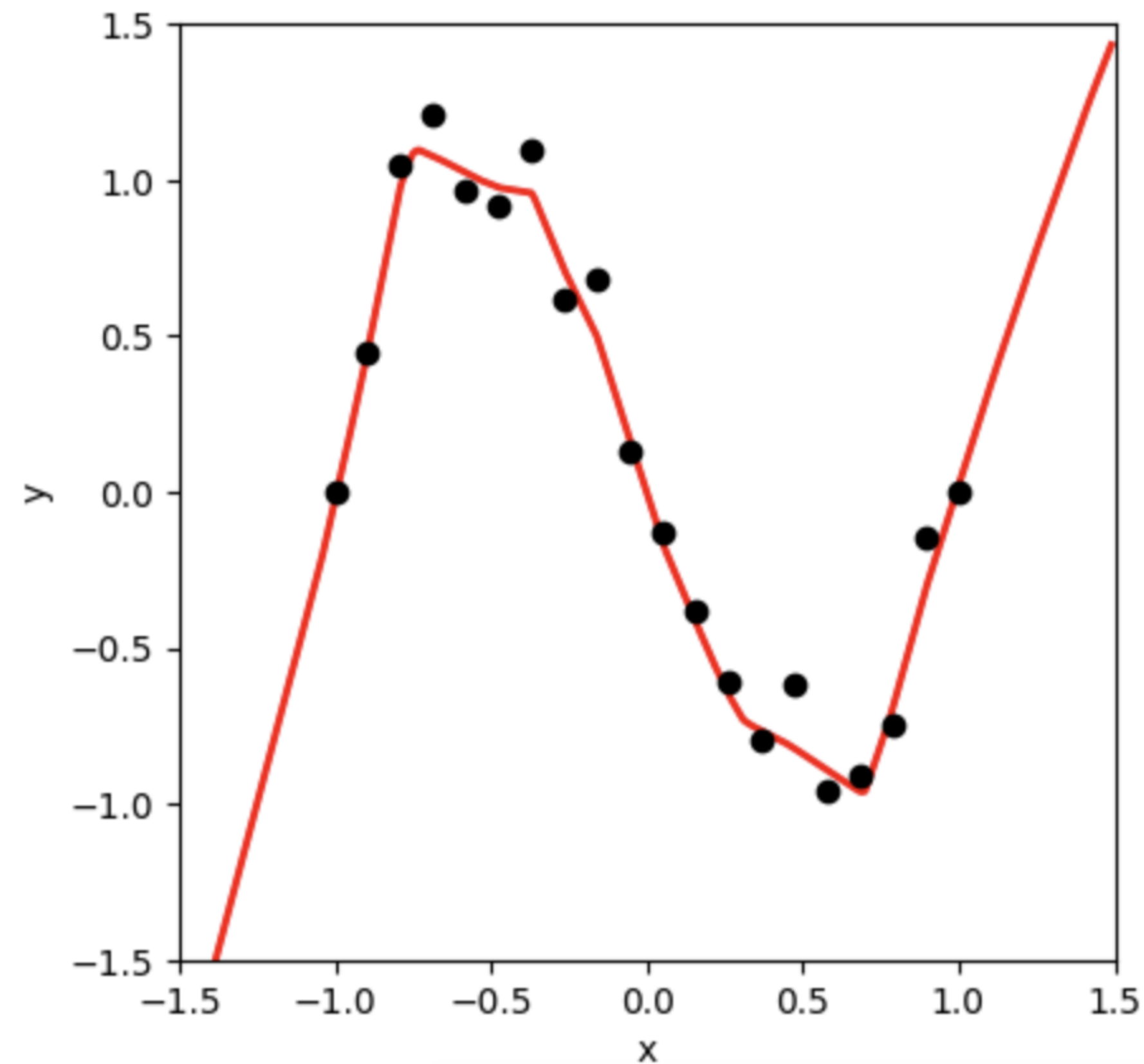


[Henighan*, Kaplan*, Katz*, et al., 2020]

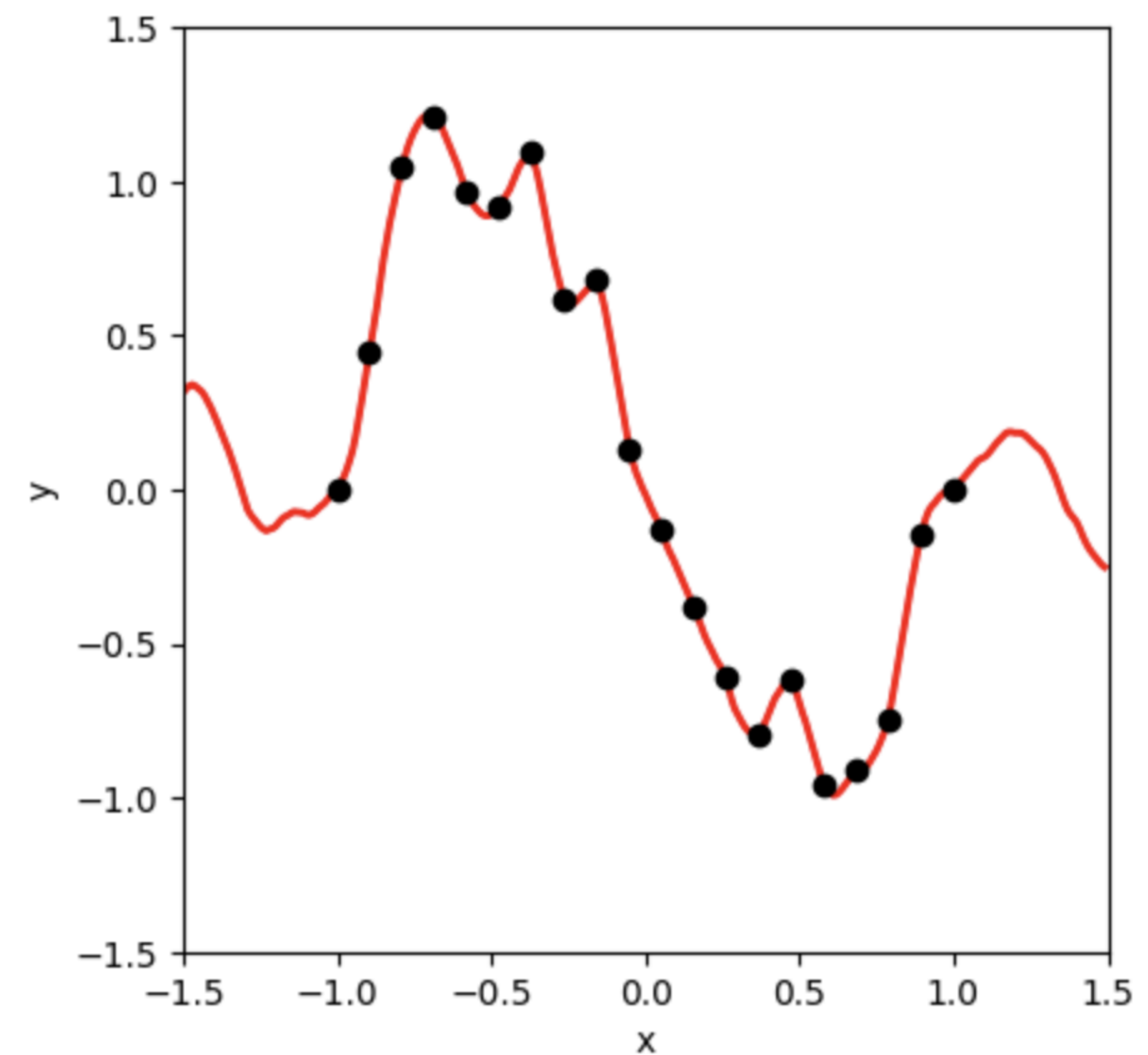
Answer: there is an optimal "aspect ratio" (width/depth), which varies for different problems. But large range of aspect ratios are acceptable. Depth and width should be roughly similar: if you scale one, scale the other too.

Preview: better architectures can approximate important function classes more efficiently.

MLP



MLP $\circ \psi$



"Positional encoding" $\psi(x) = [\sin(2^k x), \cos(2^k x)]_{k=0}^{K-1}$

Preview: better architectures can approximate important function classes more efficiently.

Goal: Fit this image
(a function $x,y \rightarrow I$)

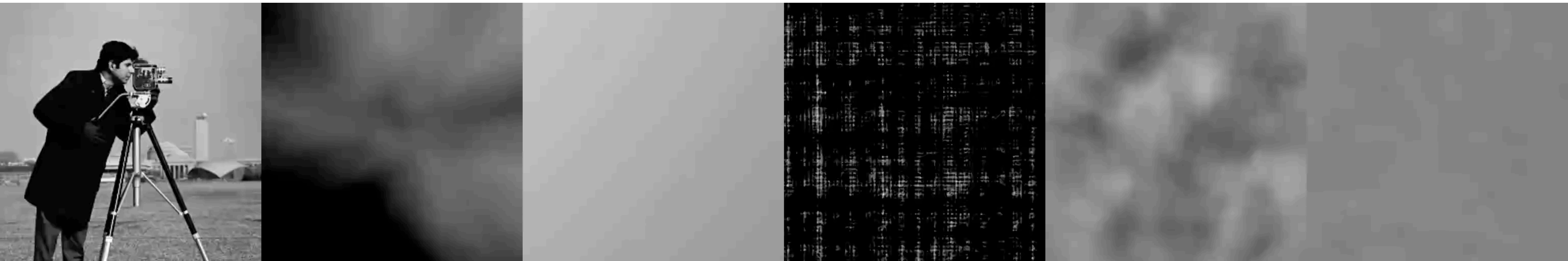
ReLU-net

TanH-net

ReLU P.E.

RBF ReLU

SIREN



(Note: this result may be due to improved approximation ability but it might also be due to improved optimization ability; these two effects are typically coupled in experiments)

[Sitzmann*, Martel*, Bergman, Lindell, Wetzstein, NeurIPS 2020]

Preview: better architectures *are not necessarily universal*

- Being universal has a cost: we can fit any kind of data, including noise
- Sometimes it is better to impose architectural constraints to prevent overfitting to noise, or to bake in structural knowledge we know a priori is true
- This is what next week's lectures will be about

Summary and comments

- Main idea: approximate a function by a sum of “basis” functions
- Tells us fundamental limitations, with practical implications
- In practice, can seemingly do with much smaller networks...
- Shallow networks cannot easily approximate deep networks (efficiency)
- Building blocks for understanding approximation with other architectures